

PCITL408

OPERATING SYSTEMS LAB



DEPARTMENT OF INFORMATION TECHNOLOGY
GOVERNMENT ENGINEERING COLLEGE PALAKKAD
SREEKRISHNAPURAM, PALAKKAD – 678633

VISION OF THE INSTITUTION

Excellence through the wings of science and technology

MISSION OF THE INSTITUTION

To transform youth to talented engineers with creativity and integrity who can meet the technological challenges for the service of society.

VISION OF THE DEPARTMENT

To achieve excellent standards in IT education and research by keeping abreast of innovations in Information Technology

MISSION OF THE DEPARTMENT

- **To nurture and develop students as competent IT professionals capable of undertaking challenging and innovative programmes.**
- **To foster self disciplined and socially committed entrepreneurs.**

LIST OF EXPERIMENTS

Sl No:	Name of the Experiment	Page No:
1	Familiarization of system calls (fork, exec, getpid, exit, wait, close, stat etc) in operating system and Demonstration of creating partition and installing OperatingSystem	
2	Implementation of Process and thread (Life cycle of process): (i) Process creation and Termination; (ii) Thread creation and Termination	
3	Implement process scheduling algorithms (FCFS, SJF, Round Robin, Priority) and compute average waiting time and average turn-around time.	
4	Inter-process communication using pipes, message queues and shared memory.	
5	Implement Producer-Consumer Problem and solve the problem using Semaphores	
6	Implementation of Dining Philosophers' Problem using threads and semaphores	
7	Implementation of banker's algorithm	
8	Implement memory management schemes (first fit, best fit and worst fit)	
9	Implement page replacement algorithms: FIFO, LRU and Optimal	
10	Implementation of Disk Scheduling using FCFS, SCAN, C-SCAN, LOOK, C-LOOK and SSTF algorithm	
11	Implement File Organization Technique (Single Level and Multi-Level Directory) and Allocation Strategies (Sequential, Linked and Indexed)	
12	Familiarization of Cloud Platforms and Virtualization tools(eg:- VMWare)	

EXPERIMENT – 1**FAMILIARIZATION OF SYSTEM CALLS****AIM:**

- To familiarize the system calls: fork(), exec(), getpid(), exit(), wait(), stat(), close()etc
- Demonstration of creating partition and installing Operating System

THEORY:

System Calls: System Call is an interface to the service made available by an Operating System.

- **fork():** fork() is used to create a separate duplicate process. It takes no arguments and returns a process id. It is defined in unistd.h

Syntax: **pid_t fork();**

New process is called child process and the process made by fork() is called the Parent process. Both processes run concurrently. After a new child process is created, both processes will execute from the next instruction following the fork() system call. It returns an integer value

$$\text{Return Value of fork() = } \begin{cases} -\text{ve if creation of a child process was unsuccessful} \\ 0 \text{ to the newly created child process} \\ +\text{ve to the parent process and it is the process id} \\ \text{of the child process} \end{cases}$$

The returned process id is of type pid_t , defined in sys/ types.h.

- **getpid():** Returns the process id of the calling process.

Syntax: **pid_t getpid();**

- **exec():** exec family of functions replaces the current running process with a new process. The process id will not be changed. It is defined in unistd.h. exec family contains following system calls.

execl (), execlp (), execl (), execv (), execvp (), execvpe ()

Syntax

#include <unistd.h>

extern char **environ;

int execl(const char *pathname, const char *arg, .../*, (char *) NULL */);

int execlp(const char *file, const char *arg, .../*, (char *) NULL */);

```

int execl(const char *pathname, const char *arg, .../*, (char *) NULL, char *const envp[]
*/);
int execv(const char *pathname, char *const argv[]);
int execlp(const char *file, char *const argv[]);
int execvp(const char *file, char *const argv[], char *const envp[]);

```

The list of arguments must be terminated with a NULL POINTER. These functions have the same base 'exec' followed by one or more letters. □

e : It is an array of pointers that points to environment variables and is passed explicitly to the newly loaded process.

l : command line arguments passed a list to the function.

p : path environment variable which helps to find the file, to be loaded into process, passed as an argument.

v : command line arguments are passed as a vector of pointers (ie Array of pointers)

It returns only if an error has occurred. The return value is 1 and error number is set to indicate the error. `exec()` is used to execute a program within a process. The program can be a command or executable binary file.

- `wait()`: `wait()` system call waits for the child process to exit or a signal is received. After child process terminates, parent continues its execution after wait system call instruction.

Child process may terminate due to any of these:

- i) It calls `exit()`
- ii) It returns from main
- iii) It receives a signal (from the OS or another process) whose default action is to terminate.

Syntax: `pid_t wait(int * stat_loc);`

It returns the process id of the dead children.

If any process has more than one child processes, then after calling `wait()`, the parent process has to be in wait state if no child terminates.

- If only one child process is terminated, then `wait()` returns process ID of the terminated child process.

- If more than one child processes are terminated, then wait() reap any arbitrary child and return process ID of that child process.

If any process has no child process, then wait() returns immediately “1”.

- exit(): exit() terminates the process normally. The status value is returned to the parent process.

Syntax: **void exit(int status);**

- stat(): stat() is used to get information about the file and file system. It is defined in the header file sys/ stat.h. In the header file, there is a structure named as stat which can store the details of a file.

stat structure:

```
struct stat
{
    dev_t dev; //id of device containing file
    ino_t ino; //inodenumbr
    mode_t mode; //File type and mode
    nlink_t nlink; //No: of hard links
    uid_t uid; //User id of owner
    gid_t gid; //Group id of owner
    dev_t rdev; //Device id(if special file)
    off_t size; //Total size in bytes
}
```

- open(), read(), write(), lseek (),close()
 - open(): open/ create a file.

Syntax:

```
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
int open(constchar *path, int flags[,int mode]);
```

Flags:

- O_RDONLY: read only
- O_WRONLY: write only
- O_RDWR: read and write
- O_CREAT: create file if it does not exists
- O_EXCL: prevent creation if it already exists

Mode: S_IRWXU: Read, Write, Execute permission for User

- write(): write a file

Syntax:

```
#include<fcntl.h>
size_t write(int fd , void buf , size_t count);
```

where fd is the file descriptor

- read(): read from a file

Syntax:

```
#include<fcntl.h>
size_t read(int fd , void buf , size_t count);
```

- lseek (): reposition read/write offset

Syntax:

```
#include <sys/types.h>
#include unistd.h
off_t lseek (int fd , off_t offset , int whence);
```

- close(): close a file

Syntax:

```
#include<fcntl.h>
int close(intfd);
```

The lseek () function repositions the offset to the argument **offset** according to the whence. If the whence is SEEK_SET, then the file offset is set to **offset** bytes. The return value of close(): 0 on success and -1 on error.

Demonstration of creating partition and installing Operating System:

In this tutorial, we will guide you through the steps required to install Ubuntu Desktop on your laptop or PC. This tutorial uses the latest Ubuntu 24.04 LTS release (available on April 25th 2024), which features the new Ubuntu Desktop installer.

Create a bootable USB stick

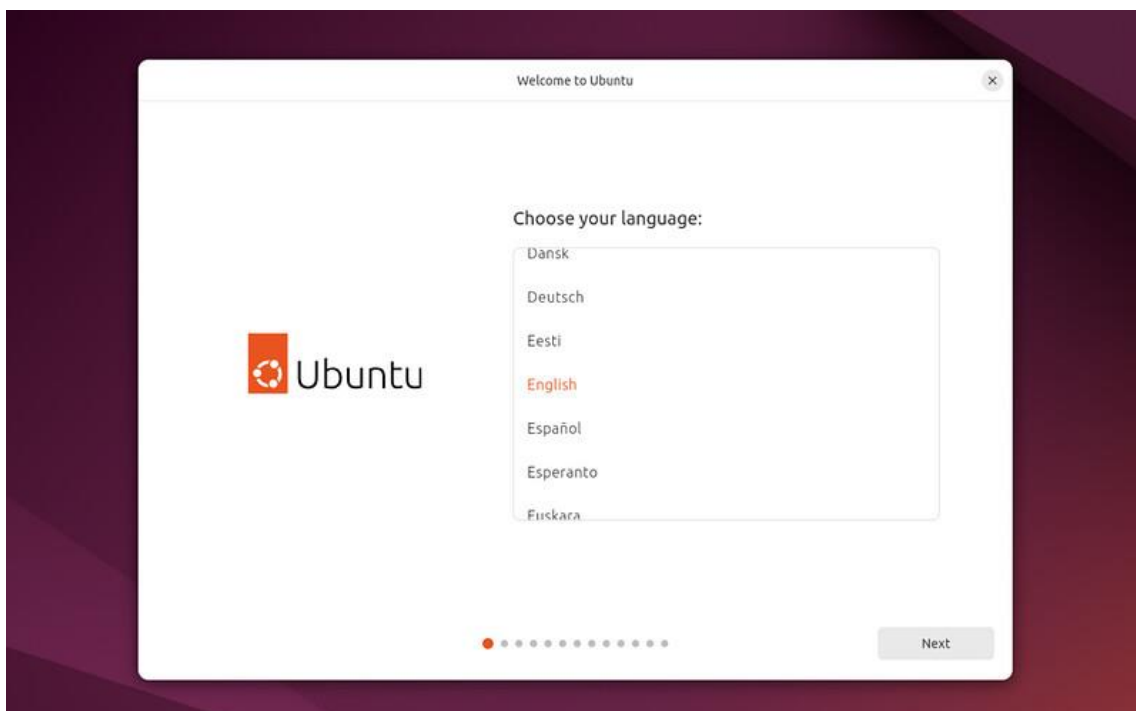
To install Ubuntu Desktop, write your downloaded ISO to a USB stick to create the installation media. This is not the same as copying the ISO and requires some special software.

Boot from USB flash drive

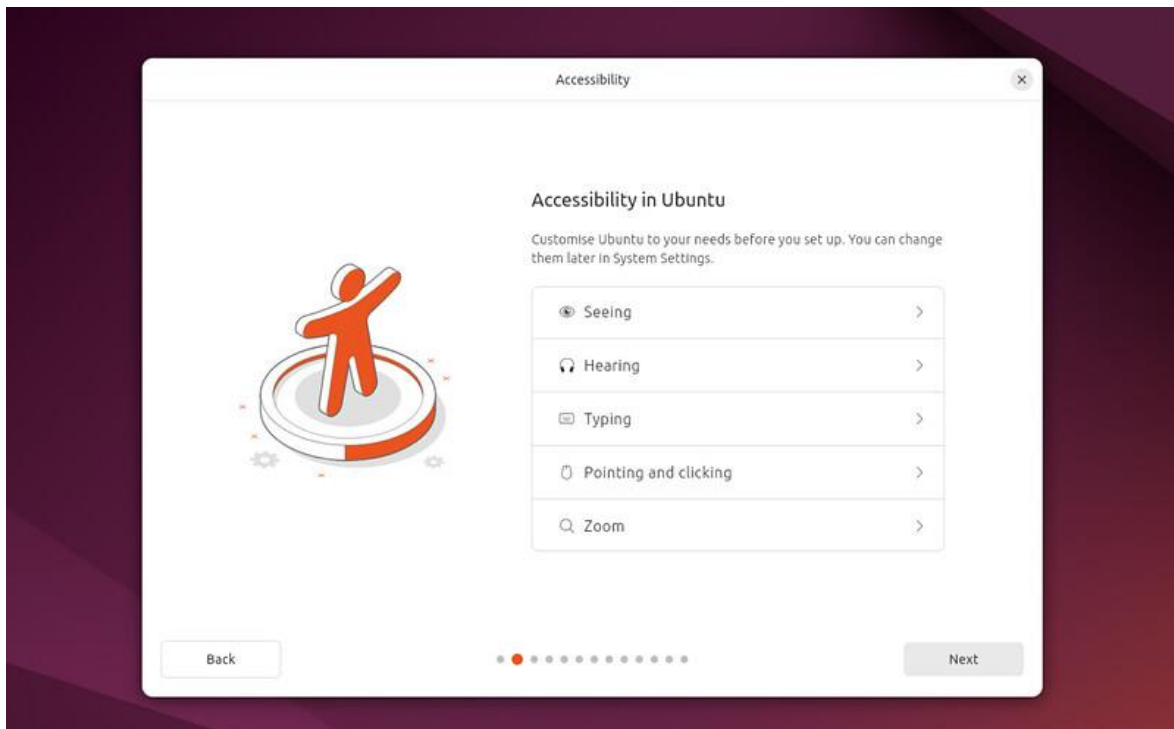
Insert the USB flash drive into the laptop or PC you want to use to install Ubuntu and boot or restart the device. It should recognise the installation media automatically. If not, try holding F12 during start up and selecting the USB device from the system-specific boot menu.

F12 is the most common key for bringing up the system boot menu but Escape, F2 and F10 are common alternatives. If unsure, look for a brief message when your system starts, this often informs you which key to press to access the boot menu.

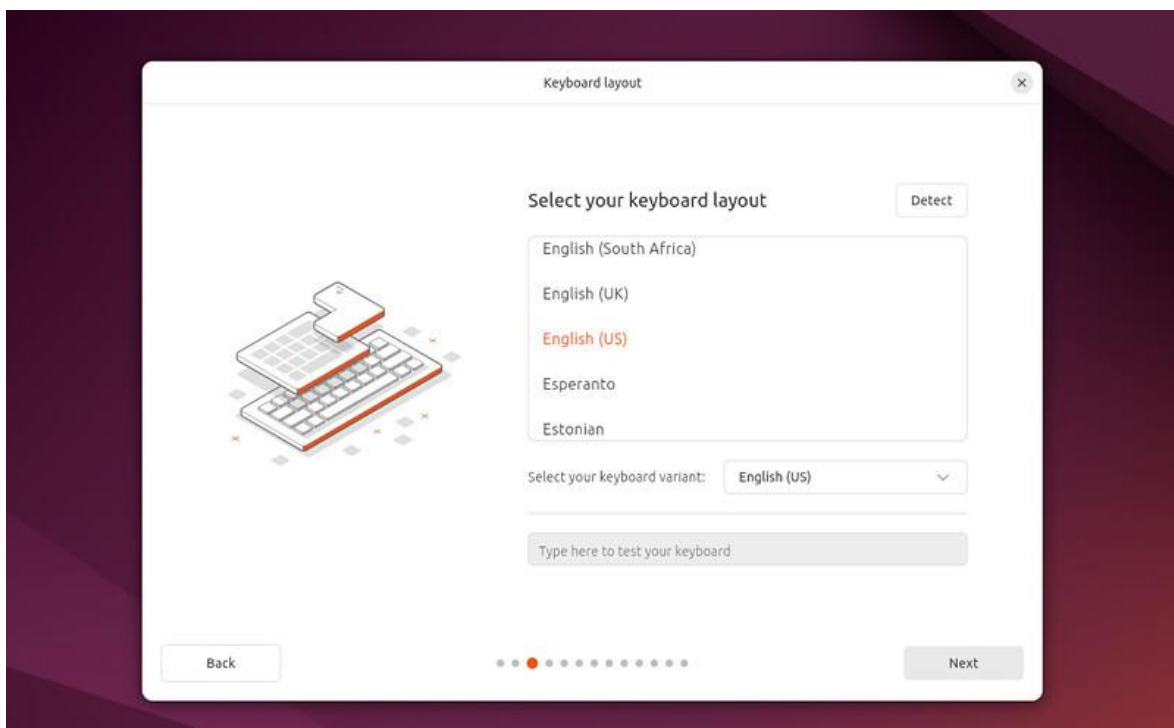
Once the installer has initialised, you are invited to choose your language.



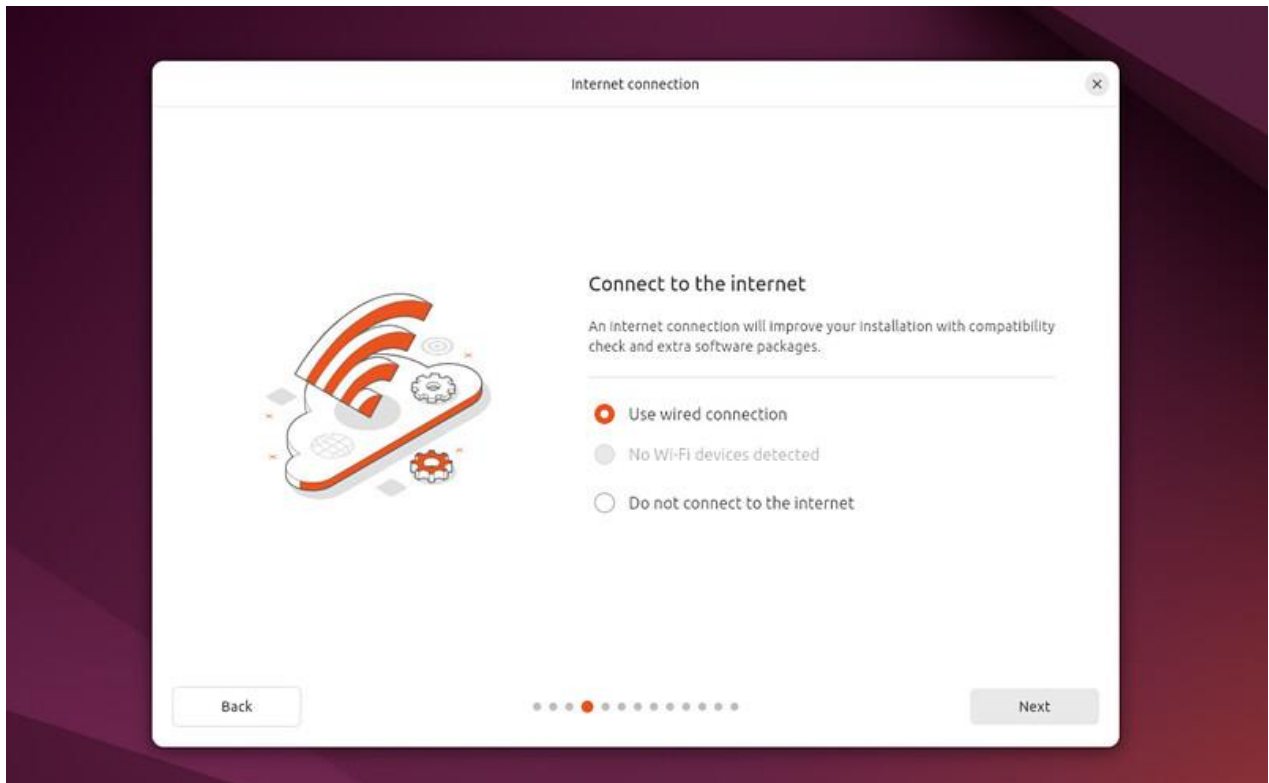
And then presented with the option to select any accessibility settings your require.



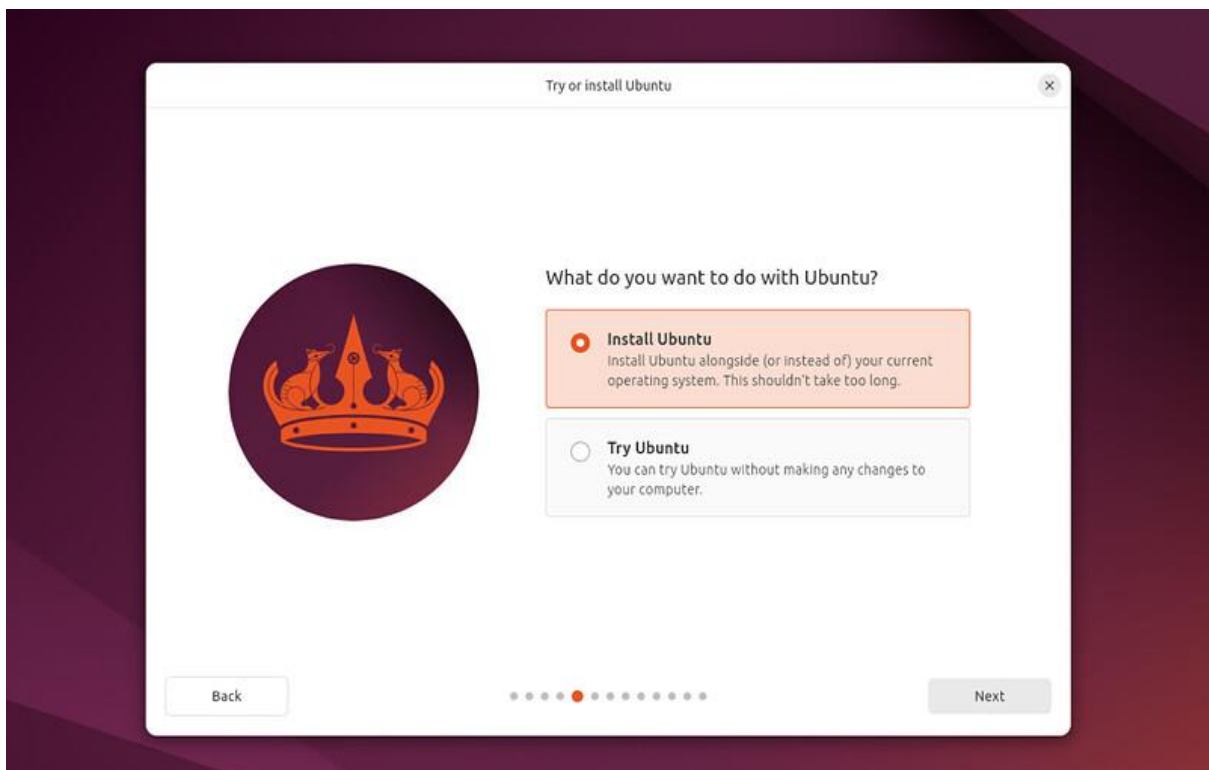
Your keyboard layout:



And connect to your network. This allows Ubuntu to download updates and third party drivers (such as NVIDIA graphics drivers) during installation.



You are then offered the choice to try or install Ubuntu.

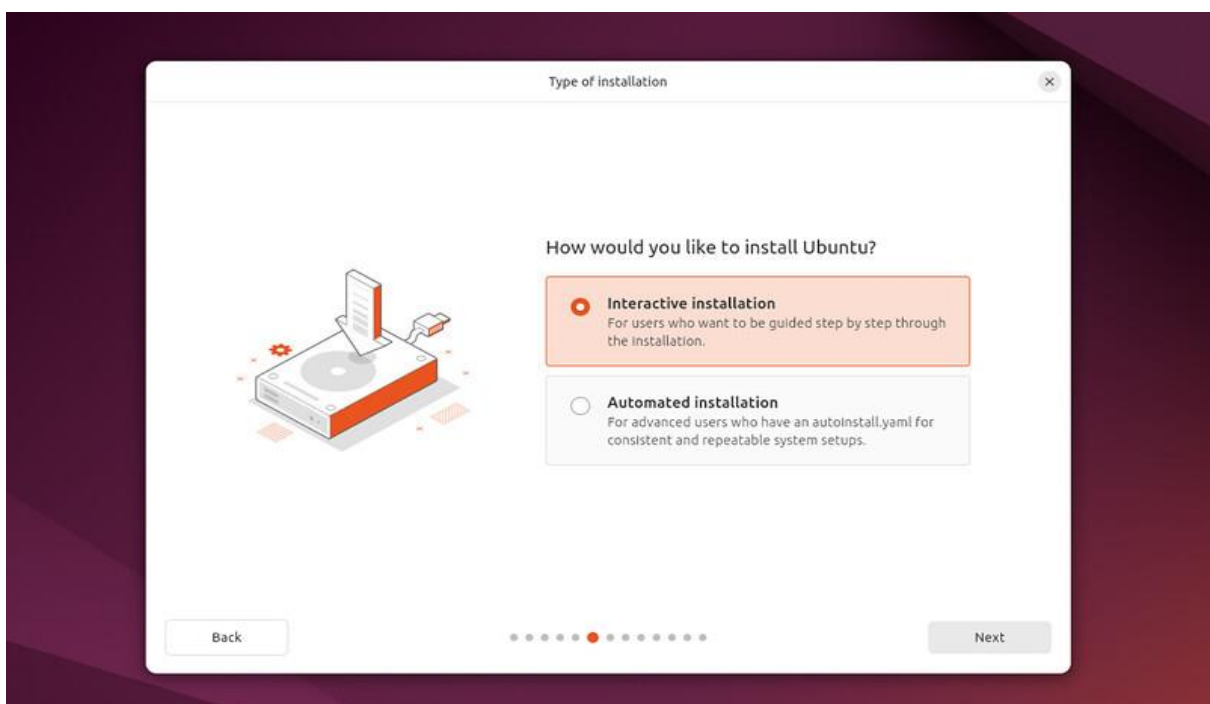


If you click Try Ubuntu, you can preview Ubuntu without making any changes to your PC. You can return to the installer menu at any time by clicking the Install Ubuntu shortcut on the desktop.

To proceed, click Install Ubuntu.

Installation setup

You will be prompted to choose between Interactive installation and Automated Installation. The interactive option is the standard route, but more advanced users can use the automated installation option to import a configuration file from a web server to standardise multiple installs and add further customisations. An example tutorial for automated installation is available [here](#).

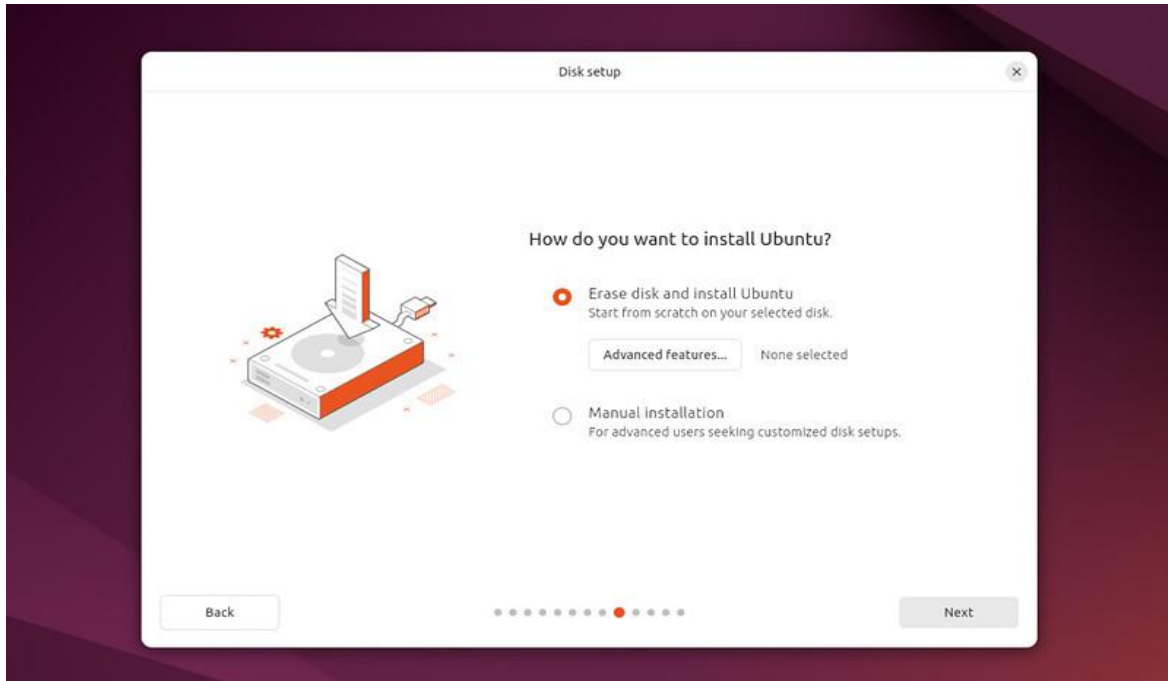


In this tutorial we will remain on the primary route.

You will be prompted to choose between the Default selection and extended selection options. The default installation comes with the basic essentials to get started which you can then expand on after install using the App Centres. The extended selection contains additional office tools and utilities, useful for offline situations.

This screen allows you to configure your installation. If you would like Ubuntu to be the only operating system on your hard drive, select **Erase disk and install Ubuntu**.

If your device currently has another operating system installed, you will receive additional options to install Ubuntu alongside that OS rather than replacing it.

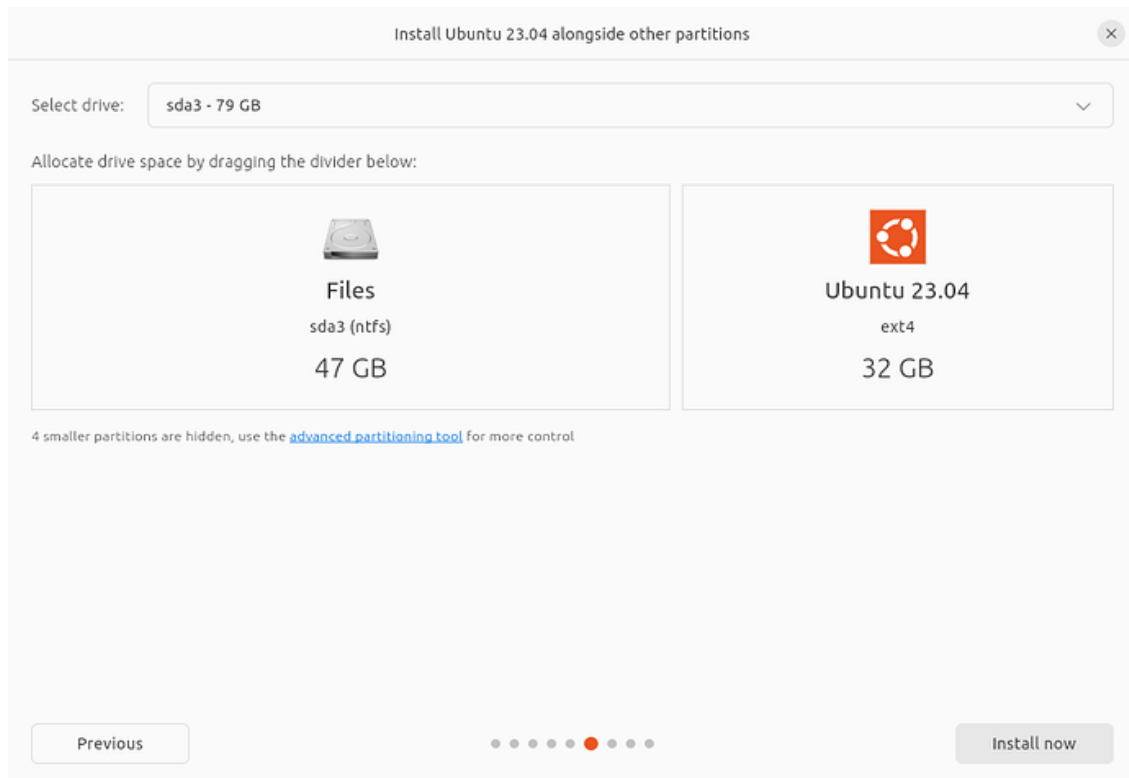


Let's take a moment to review all of the above options in detail.

Installing Ubuntu alongside another operating system

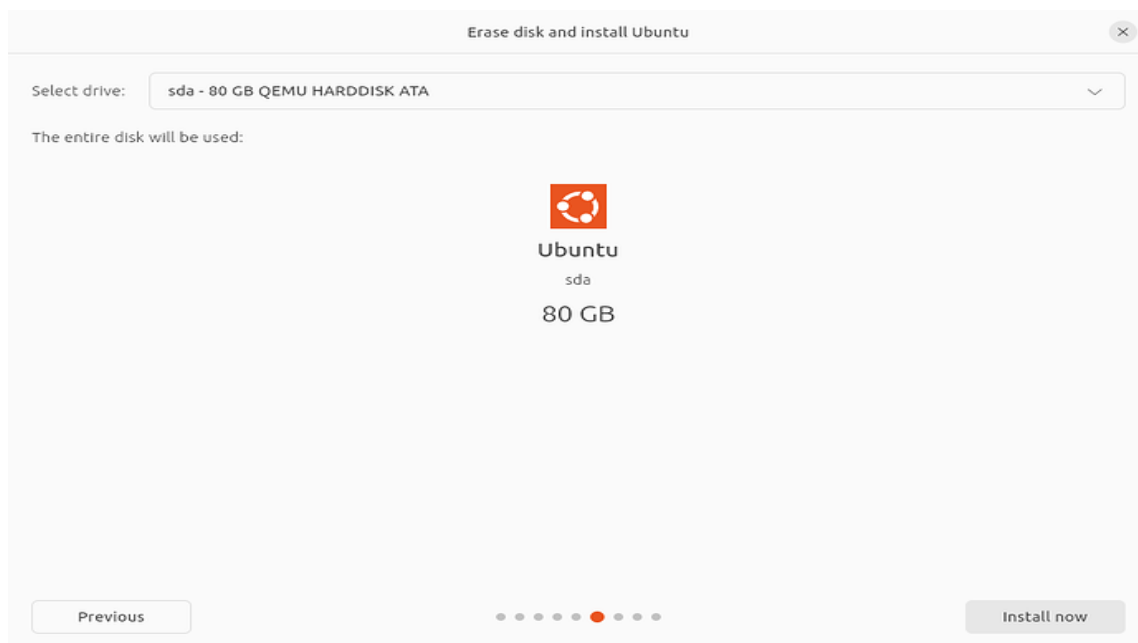
If you select this option you will be given a simple interface that allows you to select the drive you want to install Ubuntu on and a slider to determine the amount of disk space you would like Ubuntu to use. The available space is limited by the existing contents of the disk and is designed to avoid overwriting existing files.

This view automatically selects the largest partition on the drive. For more fine-grained control you can switch to the *Manual partitioning* option that is detailed further down.

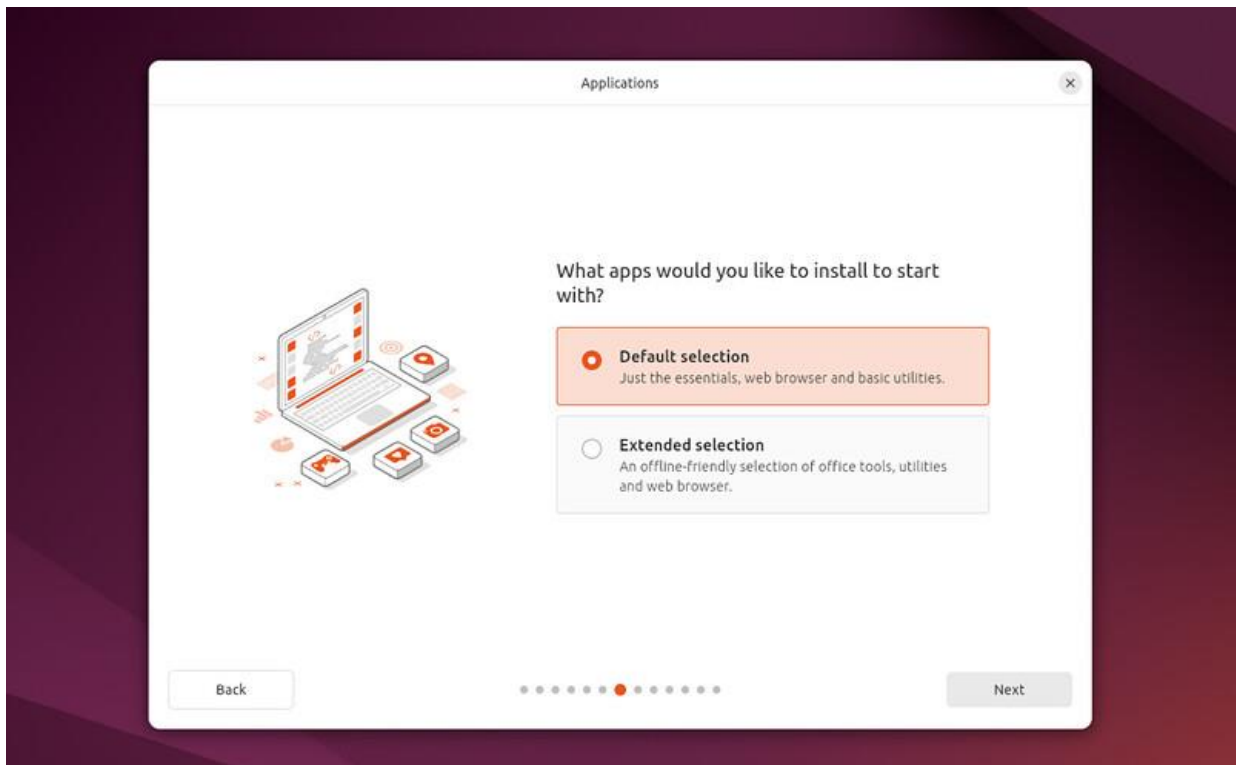


Erase disk and install Ubuntu

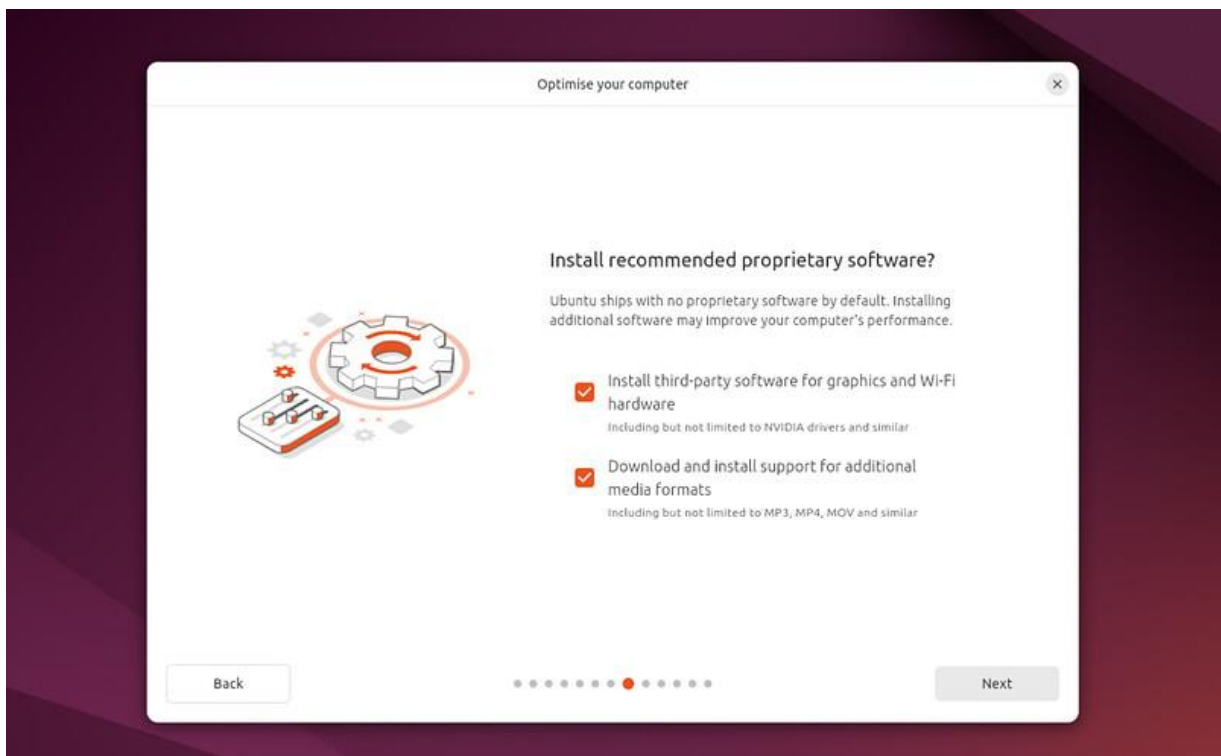
If you select this option Ubuntu will take up the entire disk space on the selected drive.



If your PC has multiple hard drives then this option allows you to install Ubuntu alongside an existing OS as long as they each have their own drive. Take care to ensure that you are selecting the right drive in this instance!



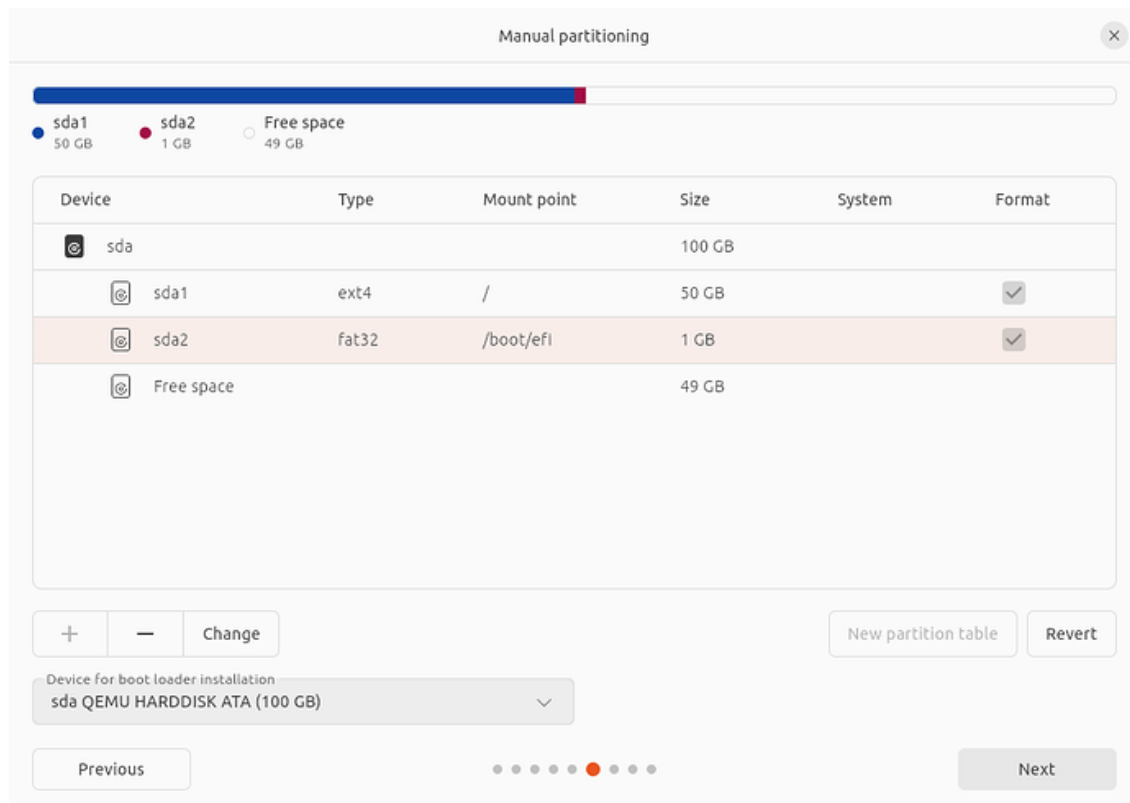
In the following screen you will be prompted to install third-party software that may improve device support and performance (for example, Nvidia graphics drivers) and support for additional media formats. It is recommended to check both of these boxes.



Manual partitioning

Manual partitioning is designed for advanced users who want to create specific configurations for their use-cases. As such we assume that these users will be comfortable with this interface and will not go into detail during this tutorial on specific setups.

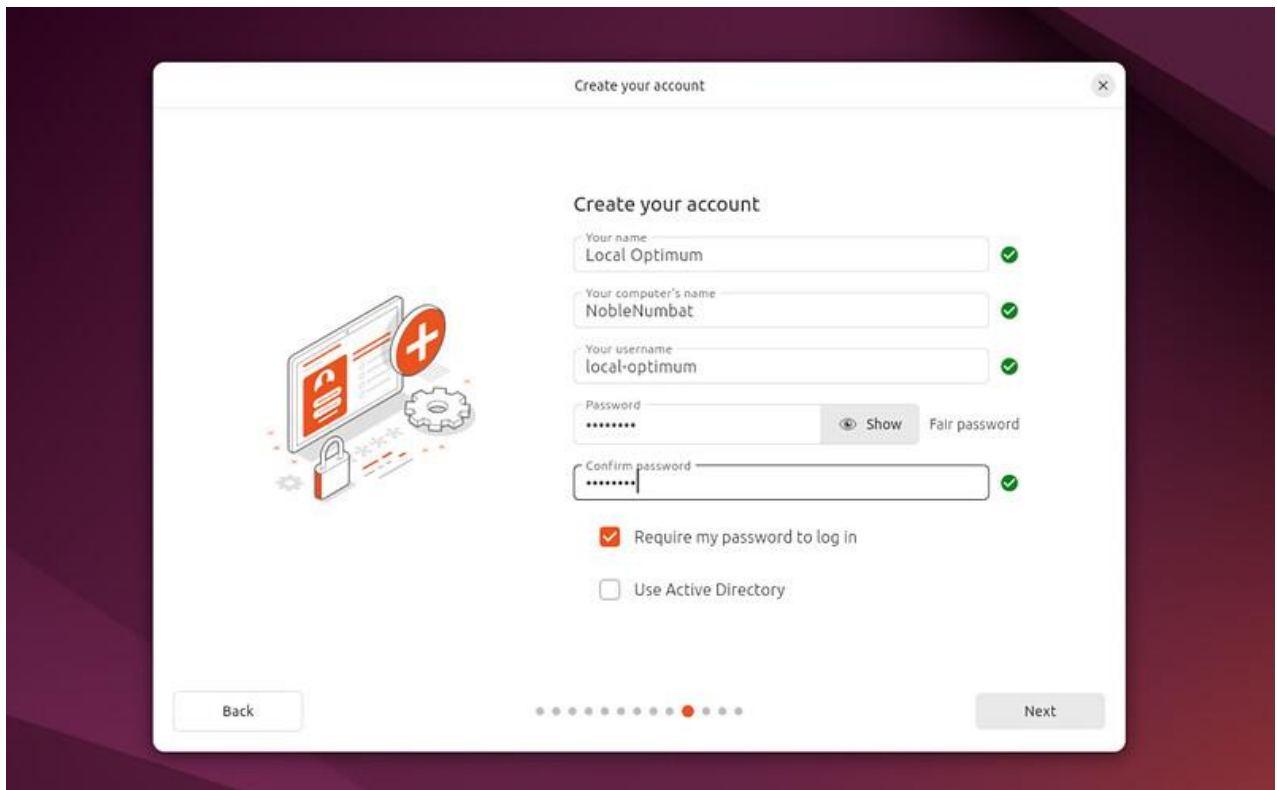
Here users can see all existing drives and partitions and create and manage new partition tables and configurations.



Create Your Login Details

On this screen, you will be prompted to enter your name and the name of your computer as it will appear on the network. Finally, you will create a username and a strong password.

You can choose to log in automatically or require a password. If you are using your device whilst travelling, it's recommended to keep "Require my password to log in" enabled.



Choose your Location

Select your location and time zone from the map screen and click **Continue**. This information will be detected automatically if you are connected to the internet.

Ready to install

Clicking next will take you to a summary of your installation configuration to give you a chance to confirm your setup before clicking Install. Once the installation has completed, you will be prompted to restart your machine.

Click **Restart Now**.

EXPERIMENT - 2**Implementation of Process and thread (Life cycle operations)****AIM:**

Implementation of Process and thread (Life cycle of process):

- (i) Process creation and Termination;
- (ii) Thread creation and Termination

THEORY:**Process**

A program in this state of execution is what we call a **process**.

The operating system manages all processes and allocates a memory zone to each of them. Each has its own stack, its own heap, its own instruction pointer, etc. This means that a process cannot easily access another process' memory or instructions. Each process is therefore a distinct entity.

We can see here that each process is assigned a **PID** (**P** rocess **ID** entifier), a non-negative integer by which we can uniquely identify a process. We can also check any process PID with the `pidof` command.

1. Creating a Process

Fork system call is used for creating a new process, which is called child process. The child process runs concurrently with the process that makes the **fork()** call (parent process). After a new child process is created, both processes will execute the next instruction following the `fork()` system call. A child process uses the same PC (Program Counter), same CPU registers, same open files which use in the parent process. It takes no parameters and returns an integer values returned by `fork()` as given below.

- Negative Value: creation of a child process was unsuccessful.
- Zero: Returned to the newly created child process.
- Positive value: Returned to parent or caller. The value contains process ID of newly created child process.

2. Exit System Call

An Exit system call has to update/clear the OS data structures of the terminating process and detach the memory pages allocated to the process from its address space. The page table entries are invalidated. There may be other processes waiting for this process to terminate. Exit system call must wake up these processes. (Currently, processes do not wait; we will see this in the next stage.) Exit has to close the files and release the semaphores acquired by the process. Per-process resource table has to be invalidated. Finally, Exit has to set the state of the process to TERMINATED. These tasks are done by invoking the **Exit Process** function

of the [process manager module](#). The end of every process, exit system call is invoked to terminate the process.

Threads

A thread is a basic unit of CPU utilization. A thread is a single sequential flow of control within a program. A process may contain one or more threads. All threads within a process share the same address space. Thread is also known as a Light Weight Process.

In a Unix/Linux operating system, the C/C++ languages provide the POSIX thread(pthread) standard Application program Interface(API) for all thread related functions. It allows us to create multiple threads for concurrent process flow. Thread operations include creation, termination, synchronization, scheduling, data management and process interaction. Pthread functions return "0" if OK.

We must include the pthread.h header file at the beginning of the script to use all the functions of the pthreads library. To execute the c file, we have to use the -pthread or -lpthread in the command line while compiling the file.

gcc-pthreadfile.c or g++ -lpthreadfile.c

The functions defined in the pthreads library include:

❖ **pthread_create:** used to create a new thread

Syntax:

```
#include<pthread.h>
int pthread_create(pthread_t
*thread, const pthread_attr_t*attr, void
(*start_routine)(void *), void *arg);
```

Parameters:

- **thread:** pointer to the thread id of the thread created.
- **attr:** pointer to a structure that is used to define thread attributes like detached state, scheduling policy, stack address, etc. Set to NULL for default thread attributes.
- **start_routine:** pointer to a subroutine that is executed by the thread. The return type and parameter type of the subroutine must be of type void *. The function has a single attribute but if multiple values need to be passed to the function, a struct must be used.
- **arg:** pointer to void that contains the arguments to the function defined in the earlier argument

❖ **pthread_exit:** used to terminate a thread

Syntax:

```
void pthread_exit(void *retval);
```

Parameters: This method accepts a mandatory parameter **retval** which is the pointer to an integer that stores the return status of the thread terminated. The scope of this

variable must be global so that any thread waiting to join this thread may read the return status.

❖ **pthread_join:** used to wait for the termination of a thread.

Syntax:

```
int pthread_join(pthread_t thread, void **thread_return);
```

Parameters: This method accepts following parameters:

- **thread:** thread id of the thread for which the current thread waits.
- **thread_return:** pointer to the location where the exit status of the thread mentioned in thread is stored.

❖ **pthread_self:** used to get the thread id of the current thread.

Syntax:

```
pthread_t pthread_self(void);
```

ALGORITHM:

1. Start
2. Create athread function for printing numbers from 1 to 5 using a loop. After printing each number, sleep
3. In the main function:
 - a. Create a thread and assign it to run the thread function and wait for the thread to finish execution (using pthread_join).
 - b. Print numbers from 500 to 504. After printing each number, sleep.
4. Stop

EXPERIMENT - 3A**FCFS SCHEDULING****AIM:**

To write a C program to implement the CPU scheduling with FIRST COME FIRST SERVE algorithm.

THEORY:

CPU scheduler will decide which process should be given the CPU for execution. It uses different algorithm to choose among the processes. First algorithm is First Come First Serve (FCFS) Algorithm.

FCFS:

In this algorithm, the process which arrive first is given the CPU first. After finishing the first process, the next process will be executed, then the next and so on.

ALGORITHM:

Create a structure for a process containing pid, burst time, waiting time and turnaround time.

Step 1: Start

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process, assign the process id and accept the CPU burst time

Step 4: Set the waiting time of the first process as '0' and the turn around time as its burst time.

Step 5: For each process in the Ready Queue calculate

(a) Waiting time for process(n)= waiting time of process (n-1) + Burst time of process(n-1)

(b) Turnaround time for Process(n)= waiting time of Process(n)+ Burst time for process(n)

Step 6: Calculate

(a) Average waiting time = Total waiting Time / Number of processes

(b) Average Turnaround time = Total Turnaround Time / Number of processes

Step 7: Stop the process

OUTPUT:**Output 1:**

Enter the number of processes: 3

Enter Burst Time for Process 0: 24

Enter Burst Time for Process 1: 3

Enter Burst Time for Process 2: 3

Process	Burst Time	Waiting Time	Turn Around Time
---------	------------	--------------	------------------

P0	24	0	24
P1	3	24	27
P2	3	27	30

Average Waiting Time: 17.000000

Average Turnaround Time: 27.000000

Output2:

Enter the number of processes: 3

Enter Burst Time for Process 0: 24

Enter Burst Time for Process 1:-3

Invalid Input

EXPERIMENT - 3B**SJF SCHEDULING****AIM:**

To write a C program to implement the CPU scheduling with SJF Scheduling algorithm.

THEORY:

CPU scheduler will decide which process should be given the CPU for execution. It uses different algorithm to choose among the processes. Next algorithm is Shortest Job First (SJF) Algorithm.

SJF:

The length of the process's next CPU burst is associated with each process. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If the next CPU bursts of two processes are the same, FCFS scheduling is used.

ALGORITHM:

Create a structure for a process containing pid, burst time, waiting time and turnaround time.

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Queue, assign the process id and accept the CPU burst time

Step 4: Sort the processes in the ready queue according to the CPU burst time (lowest to highest burst time).

Step 5: Set the waiting time of the first process as '0' and the turnaround time as its burst time.

Step 6: for each process in the Ready Q calculate

(c) Waiting time for process(n) = waiting time of process (n-1) + Burst time of process(n-1)

(d) Turn around time for Process(n) = waiting time of Process(n) + Burst time for process(n)

Step 7: Calculate

(c) Average waiting time = Total waiting Time / Number of processes

(d) Average Turnaround time = Total Turnaround Time / Number of processes

Step 8: Stop the process

OUTPUT:**Output 1:**

Enter the number of processes: 4

Enter Burst Time for Process 0: 6

Enter Burst Time for Process 1: 8

Enter Burst Time for Process 2: 7

Enter Burst Time for Process 3: 3

Process	Burst Time	Waiting Time	Turn Around Time
P3	3	0	3
P0	6	3	9
P2	7	9	16
P1	8	16	24

Average Waiting Time -- 7.000000
Average Turnaround Time -- 13.000000

Output 2:

Enter the number of processes: 4

Enter Burst Time for Process 0: 6

Enter Burst Time for Process 1: -8

Invalid Input

EXPERIMENT - 3C**PRIORITY SCHEDULING****AIM:**

To write a C program to implement the CPU scheduling with Priority Scheduling algorithm.

THEORY:

CPU scheduler will decide which process should be given the CPU for execution. It uses different algorithm to choose among the processes. Next algorithm is Priority Scheduling Algorithm.

PRIORITY SCHEDULING ALGORITHM:

A priority number (integer) is associated with each process. The CPU is allocated to the process with the highest priority (smallest integer is the highest priority). There may be two jobs in queue with the same priority, and then FCFS approach is to be performed.

ALGORITHM:

Create a structure for a process containing pid, burst time, priority, waiting time and turnaround time.

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Queue, assign the process id and accept the CPU burst time and Priority number.

Step 4: Sort the processes in the ready queue according to the Priority number (lowest number is the highest priority).

Step 5: Set the waiting time of the first process as '0' and the turnaround time as its burst time.

Step 6: for each process in the Ready Q calculate

(e) Waiting time for process(n) = waiting time of process (n-1) + Burst time of process(n-1)

(f) Turnaround time for Process(n) = waiting time of Process(n) + Burst time for process(n)

Step 7: Calculate

(e) Average waiting time = Total waiting Time / Number of processes

(f) Average Turnaround time = Total Turnaround Time / Number of processes

Step 8: Stop the process

OUTPUT: Don't write this output in the record book.

Output 1:

Enter the number of processes: 4

Enter Burst Time and Priority Number of Process 0: 6 3

Enter Burst Time and Priority Number of Process 1: 8 2

Enter Burst Time and Priority Number of Process 2: 7 4
 Enter Burst Time and Priority Number of Process 3: 3 1

Process	Burst Time	Priority	Waiting Time	Turn Around Time
P3	3	1	0	3
P1	8	2	3	11
P0	6	3	11	17
P2	7	4	17	24

Average Waiting Time -7.750000
 Average Turnaround Time -- 13.750000

Output 2:

Enter the number of processes: 4

Enter Burst Time for Process 0: 6
 Enter Burst Time for Process 1: -8

Invalid Input

EXPERIMENT - 3D

ROUND ROBIN SCHEDULING

AIM:

To write a C program to implement the CPU scheduling with Round Robin Scheduling algorithm.

THEORY:

CPU scheduler will decide which process should be given the CPU for execution. It uses different algorithm to choose among the processes. Next algorithm is Round Robin Scheduling Algorithm.

ROUND ROBIN SCHEDULING ALGORITHM:

Time slices are assigned to each process in equal portions and in circular order, handling all processes execution. After this time has elapsed, the process is preempted and added to the end of the ready queue. This allows every process to get an equal chance.

ALGORITHM:

Create a structure for a process containing pid, burst time, remaining burst time, waiting time and turnaround time.

Step1: Start the process

Step 2: Set the execution time, total execution time, average waiting time and average turnaround time as 0.

Step 3: Accept the number of processes in the ready Queue

Step 4: For each process in the ready Queue, assign the process id and accept the CPU burst time. Set the remaining burst time of each process as its burst time.

Step 5: Accept the value of quantum time from the user.

Step 6: Start an infinite loop

Step 6.1: Set count of completed process as 0.

Step 6.2: For each process in the ready queue

Step 6.2.1: If the remaining burst time of the process is 0, increment the count of completed processes and continue with the next process.

Step 6.2.2: If the remaining burst time of the process is greater than the quantum time, set the execution time as the quantum time and calculate the remaining burst time by subtracting the quantum time from the remaining burst time.

Step 6.2.3: If the remaining burst time of the process is less than the quantum time and is greater than 0, set the execution time as the remaining burst time and set the remaining burst time as 0.

Step 6.2.4: Calculate total execution time by adding execution time to total execution time.

Step 6.2.5: Set the turnaround time of the process as the total execution time.

Step 6.3: If all the processes are completed, exit from the infinite loop.

Step 7: For each process in the Ready Queue,

(g) Waiting time for process(n) = turnaround time of process (n) - Burst time of process(n)

(h) Calculate the total turnaround time and total waiting time.

- (i) Display the pid, burst time, waiting time and turnaround time of each process.

Step 7: Calculate

(g) Average waiting time = Total waiting Time / Number of processes

(h) Average Turnaround time = Total Turnaround Time / Number of processes

Step 8: Stop the process

OUTPUT: Don't write this output in the record book.

Output 1:

Enter the number of processes 4

Enter the burst time of the process1 8

Enter the burst time of the process2 6

Enter the burst time of the process3 7

Enter the burst time of the process4 4

Enter the quantum time 2

Process	burst time	waiting time	turnaround time
P1	8	16	24
P2	6	14	20
P3	7	18	25
P4	4	12	16

Average waiting time = 15.000000

Average turn around time = 21.250000

Output 2:

Enter the number of processes: 4

Enter Burst Time for Process 1: 6

Enter Burst Time for Process 2: -8

Invalid Input

EXPERIMENT - 4A

IPC USING ORDINARY PIPES

AIM:

To write a C program to implement the Inter process communication using Ordinary Pipes.

THEORY:

Cooperating processes require an Interprocess communication (IPC) mechanism that will allow them to exchange data and information.

Two models of IPC.

- ▣ Message Passing
- ▣ Shared memory

Message-Passing Model

- Communication takes place by means of messages exchanged between the cooperating processes.

Shared Memory Model

- A region of memory that is shared by cooperating processes is established.
- Processes can then exchange information by reading and writing data to the shared region.

Pipe:

Pipe is one of the message passing strategies for communication in client server systems. Pipe acts as a conduit allowing two processes to communicate. There are two types of Pipe: Ordinary pipes and Named Pipes. Ordinary pipe can be created by a parent process and can be used to communicate with a child process that it created. Named pipe can be accessed without a parent-child relationship.

Ordinary pipes are constructed using the function

```
pipe(int fd[]);
```

This function creates a pipe that is accessed through the int fd[] file descriptors: fd[0] is the read-end of the pipe, and fd[1] is the write-end.

ALGORITHM:

- Step 1: Start.
- Step 2: Initialize a buffer for storing the message.
- Step 3: Create an Ordinary Pipe.
- Step 4: Fork a child process.
- Step 5: In parent process, write a message into the write end of the pipe.
- Step 6: In the child process, read the message from the read end of the pipe and store it into the buffer.
- Step 7: Write the message in the buffer to the screen.
- Step 8: Stop.

EXPERIMENT - 4B**IPC USING NAMED PIPES****AIM:**

To write a C program to implement the Inter process communication using Named Pipes.

THEORY:

Named pipes are referred to as FIFOs in UNIX systems. Once created, they appear as typical files in the file system.

A FIFO is created with the `mkfifo()` system call and manipulated with the ordinary `open()`, `read()`, `write()`, and `close()` system calls. The communicating processes must reside on the same machine.

Syntax:

```
#include<sys/types.h>
```

```
#include<sys/stat.h>
```

```
int mkfifo(const char *pathname, mode_t mode);
```

`pathname` is the name of the pipe. `mode` is the permissions that want to give on the pipe. `mkfifo()` system call returns 0 on successful completion; otherwise -1. Since named pipe is considered as a special file, we can use ordinary `open()`, `read()`, `write()`, and `close()` system calls for opening, reading, writing and closing a FIFO.

By default, FIFO works in blocked mode ie both sender and receiver should be present for communication.

To implement the Inter process communication using Named Pipes, three programs are needed.

Program1: To create a Named Pipe.

Program2: Sender Process.

Program3: Receiver Process.

ALGORITHM:**Sender Process**

1. Create a message.
2. Open the named pipe in write mode.
3. Write the message to the named pipe.
4. Print the sent message.

Receiver Process

1. Create a buffer for storing the message.
2. Open the named pipe in read mode.
3. Read the message from the named pipe.
4. Print the received message.

EXPERIMENT - 4C

IPC USING MESSAGE QUEUES

AIM:

To write a C program to implement the Inter process communication using Message Queues.

THEORY:

A message queue is a queue of messages stored within the kernel and identified by a message queue identifier. All processes can exchange information through access to a common system message queue. The sending process places a message onto a queue which can be read by another process. Each message is given an identification or type so that processes can select the appropriate message. Process must share a common key in order to gain access to the queue in the first place.

System Calls

- **msgget()**

A new queue is created or an existing queue is opened by using **msgget()** system call.

Syntax:

```
#include <sys/msg.h>
int msgget(key_t key, int msgflg);
```

On success, **msgget()** either returns the message queue identifier (a non-negative integer) for a newly created message queue or returns the identifier for a queue which exists with the same key value. On failure, -1 is returned.

- **msgsnd() and msgrcv()**

New messages are added to the end of a queue by **msgsnd()** system call. Every message has a positive long integer type field, a non-negative length, and the actual data bytes. Messages are fetched from a queue by **msgrcv()** system call. We can fetch messages based on their type field.

Syntax:

```
#include <sys/msg.h>
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
size_t msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg);
```

The **msgsnd()** and **msgrcv()** system calls are used to send messages to, and receive messages from, a message queue. The calling process must have write permission on the message queue in order to send a message, and read permission to receive a message.

The **msgp** argument is a pointer to a caller-defined structure of the following general form:

```
struct msgbuf {
```

```

        long mtype;          /* message type, must be > 0
*/
        char mtext[1];      /* message data */
    };

```

The **mtext** field is an array whose size is specified by **msgsz**, a nonnegative integer value. The **mtype** field must have a strictly positive integer value. This value can be used by the receiving process for message selection.

The **msgrcv()** system call removes a message from the queue specified by **msqid** and places it in the buffer pointed to by **msgp**. The argument **msgsz** specifies the maximum size in bytes for the member **mtext** of the structure pointed to by the **msgp** argument.

The **msgtyp** argument specifies the type of message requested, as follows: If **msgtyp** is 0, then the first message in the queue is read. If **msgtyp** is greater than 0, then the first message in the queue of type **msgtyp** is read.

- **msgctl()**

It performs various operations on a queue. Generally, it is used to destroy message queue. Syntax:

```

#include <sys/msg.h>
int msgctl(int msqid, int cmd, struct msqid_ds *buf);

```

msgctl() performs the control operation specified by a **cmd** on the message queue with identifier **msqid**. The **msqid_ds** data structure is defined in <sys/msg.h>.

- **ftok()**

ftok() converts a pathname and a project identifier to an IPC key. IPC key is usable in subsequent calls to **msgget()**, **semget()**, and **shmget()**. The path argument must be the path name of an existing file. The **ftok()** function returns the same key value for all paths that name the same file, when called with the same id value. If successful, **ftok()** returns a key. Otherwise, returns -1.

ALGORITHM:

Sender Process

1. Create the structure of a message with a long variable **msg_type** and a char array variable **msg_text**.
2. Generate a key for IPC.
3. Create a message Queue.
4. Set the message type.
5. Get the message from the user.
6. Send the message with message flag 0 or 1. [0 means the sending process will be suspended if the message queue is full until the space is available and 1 means the process will return immediately without sending]

7. Print the sent message.

Receiver Process

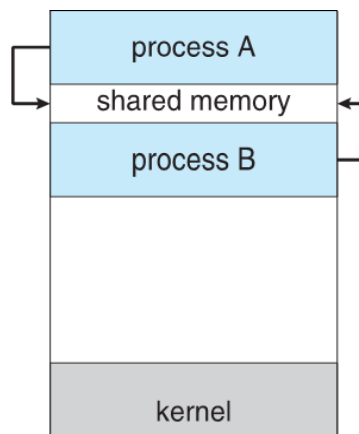
1. Create the structure of a message with a long variable msg_type and a char array variable msg_text.
2. Generate the key for IPC with the same arguments used in the Sender process.
3. Open the message Queue.
4. Receive the message with the same message type and message flag used in the Sender process.
5. Print the received message.
6. Remove the message queue.

EXPERIMENT - 4D**IPC USING SHARED MEMORY****AIM:**

To write a C program to implement the Inter process communication using Shared memory.

THEORY:**Shared Memory**

A region of memory that is shared by cooperating processes is established. Interprocess communication using shared memory requires communicating processes to establish a region of shared memory. Shared-memory region resides in the address space of the process creating the shared-memory segment. Other processes that wish to communicate must attach this shared-memory segment to their address space.



Processes can then exchange information by reading and writing data to the shared region.

Steps

- Create the shared memory segment or use an already created shared memory segment (shmget())
- Attach the process to the already created shared memory segment (shmat())
- Detach the process from the already attached shared memory segment (shmdt())
- Control operations on the shared memory segment (shmctl())

System Calls

- **ftok()**
ftok() converts a pathname and a project identifier to an IPC key. IPC key is usable in subsequent calls to msgget(), semget(), and shmget(). The path argument must be the path name of an existing file. The ftok() function returns the same key value for all paths that name the same file, when called with the same id value. If successful, ftok() returns a key. Otherwise, returns -1.

- **shmget()**

shmget() is used to obtain access to a shared memory segment.

Syntax:

```
#include <sys/ipc.h>
#include <sys/shm.h>
int shmget(key_t key, size_t size, int shmflg)
```

The first argument, key, recognizes the shared memory segment. The key can be either an arbitrary value or one that can be derived from the library function ftok(). The second argument, size, is the size of the shared memory segment. The third argument, shmflg, specifies the permissions and the required shared memory flag/s such as IPC_CREAT.

This call would return a valid shared memory identifier (used for further calls of shared memory) on success and -1 in case of failure.

- **shmat()**

shmat() is used to attach shared memory segments to the address space of the calling process.

Syntax:

```
#include <sys/types.h>
#include <sys/shm.h>
void * shmat(int shmid, const void *shmaddr, int shmflg)
```

The first argument, shmid, is the identifier of the shared memory segment. This id is the shared memory identifier, which is the return value of shmget() system call.

The second argument, shmaddr, is to specify the attaching address. If shmaddr is NULL, the system by default chooses the suitable address to attach the segment. If shmaddr is not NULL and SHM_RND is specified in shmflg, the attach is equal to the address of the nearest multiple of SHMLBA (Lower Boundary Address).

The third argument, shmflg, specifies the required shared memory flag/s such as SHM_RND (rounding off address to SHMLBA) or SHM_EXEC (allows the contents of segment to be executed) or SHM_RDONLY (attaches the segment for read-only purpose, by default it is read-write) or SHM_REMAP (replaces the existing mapping in the range specified by shmaddr and continuing till the end of segment).

This call would return the address of attached shared memory segment on success and -1 in case of failure. To know the cause of failure, check with errno variable or perror() function.

- **shmdt()**

shmdt() is used to detach shared memory segment from the address space of the calling process.

Syntax:

```
#include <sys/types.h>
#include <sys/shm.h>
int shmdt(const void *shmaddr)
```

The argument, shmaddr, is the address of shared memory segment to be detached. This call would return 0 on success and -1 in case of failure.

- **shmctl()**

shmctl() performs control operation for a shared memory segment.

Syntax:

```
#include <sys/ipc.h>
#include <sys/shm.h>
int shmctl(int shmid, int cmd, struct shmid_ds *buf)
```

The first argument, shmid, is the identifier of the shared memory segment.

The second argument, cmd, is the command to perform the required control operation on the shared memory segment. Valid values for cmd are IPC_STAT, IPC_SET, IPC_RMID, IPC_INFO and SHM_INFO. IPC_RMID marks the segment to be destroyed. The segment is destroyed only after the last process has detached it.

The third argument, buf, is a pointer to the shared memory structure named struct shmid_ds.

ALGORITHM:

Sender Process

1. Generate a key for IPC.
2. Create a shared memory using the key.
3. Attach the shared memory to the address space of the process.
4. Get the message from the user and write it in the shared memory.
5. Print the shared message.
6. Detach the shared memory from the address space.

Receiver Process

1. Generate the key for IPC with the same arguments used in the Sender process.
2. Open the shared memory using the key.
3. Attach the shared memory to the address space of the process.
4. Print the shared message.
5. Detach the shared memory from the address space.
6. Remove the shared memory segment from the system.

EXPERIMENT - 5**PRODUCER CONSUMER PROBLEM****AIM:**

To write a program to simulate producer-consumer problem using semaphores.

THEORY:

Producer consumer problem is a synchronization problem. There is a fixed size buffer where the producer produces items and that is consumed by a consumer process. One solution to the producer consumer problem uses shared memory. To allow producer and consumer processes to run concurrently, there must be available a buffer of items that can be filled by the producer and emptied by the consumer. This buffer will reside in a region of memory that is shared by the producer and consumer processes. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

Semaphore Solution (Bounded Buffer)

Use three semaphores:

- mutex = 1 binary semaphore for mutual exclusion on the buffer.
- empty = n counting semaphore tracking available empty slots.
- full = 0 counting semaphore tracking available filled slots.

Producer (pseudocode):

```
do {
    item = produce();

    wait(empty);    // ensure there is space
    wait(mutex);    // enter critical section

    insert(item);    // write into circular buffer, advance 'in'

    signal(mutex);  // leave critical section
    signal(full);    // one more filled slot available
} while (true);
```

Consumer (pseudocode):

```
do {
    wait(full);      // ensure there is an item
    wait(mutex);     // enter critical section

    item = remove(); // read from circular buffer, advance 'out'

    signal(mutex);   // leave critical section
    signal(empty);    // one more empty slot available

    consume(item);
} while (true);
```

EXPERIMENT - 6

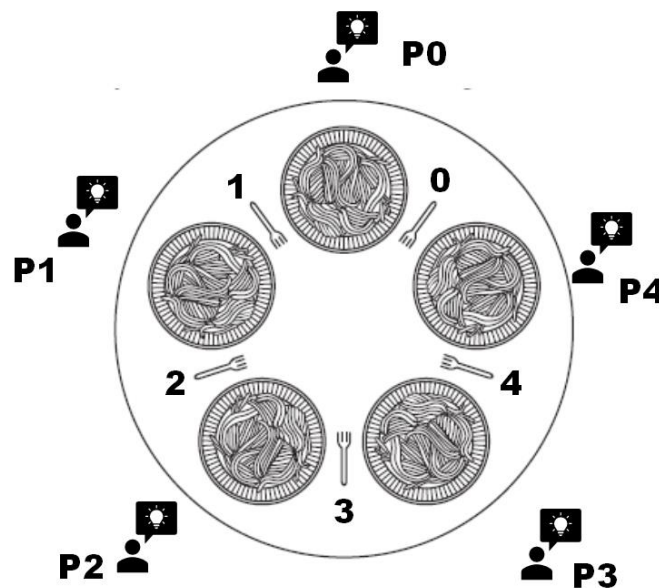
SOLVING DINING PHILOSOPHERS PROBLEM USING SEMAPHORE

AIM:

To write a C program to solve Dining Philosophers problem using Semaphore.

THEORY:**Dining Philosophers Problem**

The dining philosophers problem states that there are 5 philosophers sharing a circular table and they eat and think alternatively. There is a bowl of rice for each of the philosophers and 5 chopsticks. A philosopher needs both their right and left chopstick to eat. A hungry philosopher may only eat if there are both chopsticks available. Otherwise a philosopher puts down their chopstick and begin thinking again. When a philosopher gets hungry, he tries to pick up the two chopsticks that are closest to him (left and right). A philosopher can pick up only one chopstick at a time. One cannot pick up a chopstick that is already in the hand of a neighbour.



The dining philosopher is a classic synchronization problem as it demonstrates a large class of concurrency control problems.

Solution of Dining Philosophers Problem

A solution of the Dining Philosophers Problem is to use binary semaphore to represent a chopstick. A chopstick can be picked up by executing a **wait** operation on the semaphore and released by executing a **signal** semaphore.

The structure of the chopstick is shown below –

```
semaphore chopstick [5];
```

Initially the elements of the chopstick are initialized to 1 as the chopsticks are on the table and not picked up by a philosopher.

The structure of a random philosopher i is given as follows –

```
do {  
wait( chopstick[i] );  
wait( chopstick[ (i+1) % 5] );  
.  
. EATING THE RICE  
.  
signal( chopstick[i] );  
signal( chopstick[ (i+1) % 5] );  
.  
. THINKING  
.  
} while(1);
```

In the above structure, first wait operation is performed on chopstick[i] and chopstick[(i+1) % 5]. This means that the philosopher i has picked up the chopsticks on his sides. Then the eating function is performed.

After that, signal operation is performed on chopstick[i] and chopstick[(i+1) % 5]. This means that the philosopher i has eaten and put down the chopsticks on his sides. Then the philosopher goes back to thinking.

EXPERIMENT - 7**BANKERS ALGORITHM FOR DEADLOCK AVOIDANCE****AIM:**

To write a C program to implement Bankers Algorithm.

THEORY:

In a multiprogramming environment, several processes may compete for a finite number of resources. A process requests resources; if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other waiting processes. This situation is called a deadlock. Deadlock avoidance is one of the techniques for handling deadlocks. This approach requires that the operating system be given in advance additional information concerning which resources a process will request and use during its lifetime. With this additional knowledge, it can decide for each request whether or not the process should wait. To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process. Banker's algorithm is a deadlock avoidance algorithm that is applicable to a system with multiple instances of each resource type.

Bankers Algorithm

Consider two $n \times m$ matrices C and R where n is the number of processes and m is the number of resources. The matrix C shows how many of each resource is currently assigned to each of the processes. The matrix R shows how many resources each process still needs in order to complete its execution. There are three vectors, Existing Resources E , Possessed Resources P and Available Resources A . E gives the total no: of instances of each resource in existence, P gives the total no: of instances of each resource possessed by each process and A gives the no: of instances of resources that are currently available. Bankers Algorithm is given below.

Algorithm for checking to see if a state is safe:

1. Look for row, R , whose unmet resource needs all $\leq A$. If no such row exists, system will eventually deadlock since no process can run to completion.
2. Assume process of row chosen requests all resources it needs and finishes. Mark process as terminated, add all its resources to the A vector.
3. Repeat steps 1 and 2 until either all processes marked terminated (initial state was safe) or no process left whose resource needs can be met (there is a deadlock).

ALGORITHM:

1. START
2. Input the number of processes, np.
3. Input the number of types of resources, nr.
4. Input the Existing Resource Vector and Max matrix.
5. Clear the values in Possessed Resource Vector and Completed_Status vector.
6. Input the Current Allocation Matrix and Find the Possessed Resource Vector.
7. Find the Available Resource Vector and the Need Matrix.
8. Repeat the following steps for np times
 - a. For each incomplete process,
 - i. check whether its unmet resource needs are all less than or equal to A.
 - ii. If such a process is obtained, mark the process as completed and add it in the Execution_Sequence.
 - iii. Add the currently assigned resources of that process to Available Resource Vector.
 - b. If no such process exists, set the SAFE flag as 0 and exit from the loop.
 - c. else
 - i. If all the processes are completed, Set the SAFE flag as 1 and exit from the loop.
9. If the SAFE flag is true,
 - a. Print that the system is safe and the Execution_Sequence.
10. else
 - a. Print that the system is Unsafe.
11. STOP

SAMPLE OUTPUTS

OUTPUT1 :

```
ebey@ubuntu:~/OSLab$ gccBankers.c
ebey@ubuntu:~/OSLab$ ./a.out
```

Enter the no: of processes5

Enter the no: of types of resources 4

Enter the total no: of instances of each type of resources

No: of instances of Resource0: 6

No: of instances of Resource1: 3

No: of instances of Resource2: 4

No: of instances of Resource3: 2

Enter the maximum no: of resources required by each process

Enter the maximum no: of R0 resources required by process0 4

Enter the maximum no: of R1 resources required by process0 1

Enter the maximum no: of R2 resources required by process0 1

PCITL408 OPERATING SYSTEMS LAB

```
Enter the maximum no: of R3 resources required by process0      1
Enter the maximum no: of R0 resources required by process1      0
Enter the maximum no: of R1 resources required by process1      2
Enter the maximum no: of R2 resources required by process1      1
Enter the maximum no: of R3 resources required by process1      2
Enter the maximum no: of R0 resources required by process2      4
Enter the maximum no: of R1 resources required by process2      2
Enter the maximum no: of R2 resources required by process2      1
Enter the maximum no: of R3 resources required by process2      0
Enter the maximum no: of R0 resources required by process3      1
Enter the maximum no: of R1 resources required by process3      1
Enter the maximum no: of R2 resources required by process3      1
Enter the maximum no: of R3 resources required by process3      1
Enter the maximum no: of R0 resources required by process4      2
Enter the maximum no: of R1 resources required by process4      1
Enter the maximum no: of R2 resources required by process4      1
Enter the maximum no: of R3 resources required by process4      0

Enter the current allocation of resources required by each
process
Enter the current allocation: of R0 resources to the process0    3
Enter the current allocation: of R1 resources to the process0    0
Enter the current allocation: of R2 resources to the process0    1
Enter the current allocation: of R3 resources to the process0    1
Enter the current allocation: of R0 resources to the process1    0
Enter the current allocation: of R1 resources to the process1    1
Enter the current allocation: of R2 resources to the process1    0
Enter the current allocation: of R3 resources to the process1    0
```

PCITL408 OPERATING SYSTEMS LAB

Enter the current allocation: of R0 resources to the process2 1
Enter the current allocation: of R1 resources to the process2 1
Enter the current allocation: of R2 resources to the process2 1
Enter the current allocation: of R3 resources to the process2 0
Enter the current allocation: of R0 resources to the process3 1
Enter the current allocation: of R1 resources to the process3 1
Enter the current allocation: of R2 resources to the process3 0
Enter the current allocation: of R3 resources to the process3 1
Enter the current allocation: of R0 resources to the process4 0
Enter the current allocation: of R1 resources to the process4 0
Enter the current allocation: of R2 resources to the process4 0
Enter the current allocation: of R3 resources to the process4 0

Max Matrix

4	1	1	1
0	2	1	2
4	2	1	0
1	1	1	1
2	1	1	0

Curr_alloc matrix

3	0	1	1
0	1	0	0
1	1	1	0
1	1	0	1
0	0	0	0

Need matrix

1	1	0	0
0	1	1	2
3	1	0	0
0	0	1	0
2	1	1	0

Exist Vector

6	3	4	2
---	---	---	---

Possess Vector

5	3	2	2
---	---	---	---

Avail Vector

1	0	2	0
---	---	---	---

Process3 can complete execution with available resources

PCITL408 OPERATING SYSTEMS LAB

```
Avail Vector
2    1    2    1
Process0 can complete execution with available resources
Avail Vector
5    1    3    2
Process1 can complete execution with available resources
Avail Vector
5    2    3    2
Process2 can complete execution with available resources
Avail Vector
6    3    4    2
Process4 can complete execution with available resources
Avail Vector
6    3    4    2
The state is Safe and the sequence of process completion is   P3
    P0    P1    P2    P4
```

OUTPUT 2:

```
ebey@ubuntu:~/OSLab$ ./a.out
```

```
Enter the no: of processes5
```

```
Enter the no: of types of resources 4
```

```
Enter the total no: of instances of each type of resources
```

```
No: of instances of Resource0: 6
```

```
No: of instances of Resource1: 3
```

```
No: of instances of Resource2: 4
```

```
No: of instances of Resource3: 2
```

```
Enter the maximum no: of resources required by each process
```

```
Enter the maximum no: of R0 resources required by process0      4
```

```
Enter the maximum no: of R1 resources required by process0      1
```

```
Enter the maximum no: of R2 resources required by process0      1
```

```
Enter the maximum no: of R3 resources required by process0      1
```

```
Enter the maximum no: of R0 resources required by process1      0
```

```
Enter the maximum no: of R1 resources required by process1      2
```

```
Enter the maximum no: of R2 resources required by process1      1
```

```
Enter the maximum no: of R3 resources required by process1      2
```

```
Enter the maximum no: of R0 resources required by process2      4
```

PCITL408 OPERATING SYSTEMS LAB

```
Enter the maximum no: of R1 resources required by process2      2
Enter the maximum no: of R2 resources required by process2      1
Enter the maximum no: of R3 resources required by process2      0
Enter the maximum no: of R0 resources required by process3      1
Enter the maximum no: of R1 resources required by process3      1
Enter the maximum no: of R2 resources required by process3      1
Enter the maximum no: of R3 resources required by process3      1
Enter the maximum no: of R0 resources required by process4      2
Enter the maximum no: of R1 resources required by process4      1
Enter the maximum no: of R2 resources required by process4      1
Enter the maximum no: of R3 resources required by process4      0
Enter the current allocation of resources required by each
process
Enter the current allocation: of R0 resources to the process0    3
Enter the current allocation: of R1 resources to the process0    0
Enter the current allocation: of R2 resources to the process0    1
Enter the current allocation: of R3 resources to the process0    1
Enter the current allocation: of R0 resources to the process1    0
Enter the current allocation: of R1 resources to the process1    1
Enter the current allocation: of R2 resources to the process1    1
Enter the current allocation: of R3 resources to the process1    0
Enter the current allocation: of R0 resources to the process2    1
Enter the current allocation: of R1 resources to the process2    1
Enter the current allocation: of R2 resources to the process2    1
Enter the current allocation: of R3 resources to the process2    0
Enter the current allocation: of R0 resources to the process3    1
Enter the current allocation: of R1 resources to the process3    1
```

PCITL408 OPERATING SYSTEMS LAB

Enter the current allocation: of R2 resources to the process3 0

Enter the current allocation: of R3 resources to the process3 1

Enter the current allocation: of R0 resources to the process4 0

Enter the current allocation: of R1 resources to the process4 0

Enter the current allocation: of R2 resources to the process4 1

Enter the current allocation: of R3 resources to the process4 0

Max Matrix

4	1	1	1
0	2	1	2
4	2	1	0
1	1	1	1
2	1	1	0

Curr_alloc matrix

3	0	1	1
0	1	1	0
1	1	1	0
1	1	0	1
0	0	1	0

Need matrix

1	1	0	0
0	1	0	2
3	1	0	0
0	0	1	0
2	1	0	0

Exist Vector

6	3	4	2
---	---	---	---

Possess Vector

5	3	4	2
---	---	---	---

Avail Vector

1	0	0	0
---	---	---	---

The state is Unsafe

EXPERIMENT - 8**CONTIGUOUS MEMORY ALLOCATION SCHEMES****AIM:**

To write a C program to simulate the following contiguous memory allocation techniques

a) Worst-fit b) Best-fit c) First-fit.

THEORY:

One of the simplest methods for memory allocation is to divide memory into several fixed-sized partitions. Each partition may contain exactly one process. In this multiple-partition method, when a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process. The operating system keeps a table indicating which parts of memory are available and which are occupied. Finally, when a process arrives and needs memory, a memory section large enough for this process is provided. When it is time to load or swap a process into main memory, and if there is more than one free block of memory of sufficient size, then the operating system must decide which free block to allocate. Best-fit strategy chooses the block that is closest in size to the request. First-fit chooses the first available block that is large enough. Worst-fit chooses the largest available block.

ALGORITHM:

Memory Block can be constructed using the structure below.

```
struct BLOCK
{
    int size;
    int isallocated;
    int pallocated;
    int frag;
} block[MAX];
```

Algorithm for First Fit:

1. Start
2. Input the number of memory blocks and size of each memory block.
3. Input the number of processes and size of each process.
4. For each process,
 - a. Find the first available memory block that is large enough to accommodate the process.
 - b. Allocate the process in the memory block.
5. Display the list of memory blocks, its allocated processes and the remaining fragment size.
6. Stop.

Algorithm for Best Fit

1. Start
2. Input the number of memory blocks and size of each memory block.
3. Input the number of processes and size of each process.
4. For each process,
 - a. Find the smallest available memory block that can accommodate the process.
 - b. Allocate the process in the memory block.
5. Display the list of memory blocks, its allocated processes and the remaining fragment size.
6. Stop.

Algorithm for Worst Fit

1. Start
2. Input the number of memory blocks and size of each memory block.
3. Input the number of processes and size of each process.
4. For each process,
 - a. Find the largest available memory block that can accommodate the process.
 - b. Allocate the process in the memory block.
5. Display the list of memory blocks, its allocated processes and the remaining fragment size.
6. Stop.

SAMPLE OUTPUTS

First fit

```
ebey@ubuntu:~/OSLab$ gccfirstfit.c
ebey@ubuntu:~/OSLab$ ./a.out
```

Enter the no: of blocks 4

Enter the size of the blocks
Block0: 20

Block1: 30

Block2: 40

Block3: 50

Enter the no: of processes 5

Enter the size of the processes
Process0: 45

Process1: 25

Process2: 15

Process3: 35

Process4: 28

Process0 is allocated to block3

Process1 is allocated to block1

Process2 is allocated to block0

Process3 is allocated to block2

Block	Isallocated	Process_allocated	BlockSize	PSize	Fragment
0	1	2	20	15	5
1	1	1	30	25	5
2	1	3	40	35	5
3	1	0	50	45	5

Best Fit

ebey@ubuntu:~/OSLab\$ gccbestfit.c

ebey@ubuntu:~/OSLab\$./a.out

Enter the no: of blocks 4

Enter the size of the blocks

Block0: 10

Block1: 20

Block2: 30

Block3: 40

Enter the no: of processes 5

Enter the size of the processes

Process0: 12

Process1: 22

Process2: 35

Process3: 8

Process4: 10

Process0 is allocated to block1

Process1 is allocated to block2

Process2 is allocated to block3

Process3 is allocated to block0

Block	Isallocated	Process_allocated	BlockSize	PSize	Fragment
0	1	3	10	8	2
1	1	0	20	12	8

2	1	1	30	22	8
3	1	2	40	35	5

Worst Fit

ebey@ubuntu:~/OSLab\$ gccworstfit.c

ebey@ubuntu:~/OSLab\$./a.out

Enter the no: of blocks 4

Enter the size of the blocks

Block0: 10

Block1: 20

Block2: 30

Block3: 40

Enter the no: of processes 5

Enter the size of the processes

Process0: 25

Process1: 15

Process2: 8

Process3: 6

Process4: 10

Process0 is allocated to block3

Process1 is allocated to block2

Process2 is allocated to block1

Process3 is allocated to block0

Block	Isallocated	Process_allocated	BlockSize	PSize	Fragment
0	1	3	10	6	4
1	1	2	20	8	12
2	1	1	30	15	15
3	1	0	40	25	15

EXPERIMENT - 9**PAGE REPLACEMENT ALGORITHMS****AIM:**

Implement page replacement algorithms: FIFO, LRU and Optimal

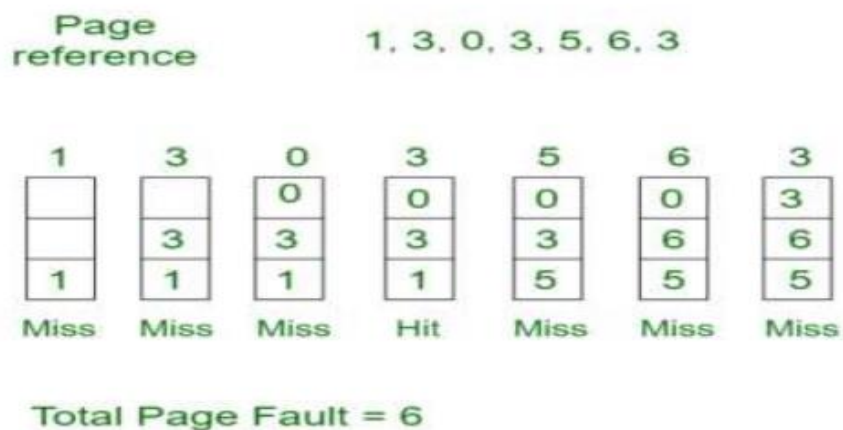
1. First In First Out (FIFO)**THEORY:**

This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced, page in the front of the queue is selected for removal.

Example

Consider page reference string 1, 3, 0, 3, 5, 6 and 3, with 3 page frames.

Find number of page faults.



- Initially all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots **3 Page Faults.**
- When 3 come, it is already in memory so **0 Page Faults.**
- Then 5 comes, it is not available in memory so it replaces the oldest page slot i.e 1. **1 Page Fault.**
- 6 comes, it is also not available in memory so it replaces the oldest page slot i.e 3. **1 Page Fault.**
- Finally when 3 come it is not available so it replaces 0. **1 page fault**

Belady's anomaly:

Belady's anomaly proves that it is possible to have more page faults when increasing the number of page frames while using the First in First out (FIFO) page replacement algorithm. For example, if we consider reference string 3,

2, 1, 0, 3, 2, 4, 3, 2,

1, 0, 4 and 3 slots, we get 9 total page faults, but if we increase slots to 4, we get 10 page faults.

ALGORITHM:

1. Start the program
2. Read the number of frames
3. Read the number of pages
4. Read the page numbers
5. Initialize the values in frames to -1
6. Allocate the pages in to frames in First in first out order.
7. Display the number of page faults.
8. Stop the program

2. Least Recently Used (LRU)

THEORY:

In this algorithm page will be replaced which is least recently used.

Example

Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 and 3 with 4 page frames. Find number of page faults.

Page reference														No. of Page frame - 4											
7	0	1	2	0	3	0	4	2	3	0	3	2	3												
			2	2	2	2	2	2	2	2	2	2	2												
		1	1	1	1	1	4	4	4	4	4	4	4												
	0	0	0	0	0	0	0	0	0	0	0	0	0												
7	7	7	7	7	3	3	3	3	3	3	3	3	3												
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit												
Total Page Fault = 6																									

Here LRU has same number of page fault as optimal but it may differ according to question.

- Initially all slots are empty, so when 7 0 1 2 are allocated to the empty slots **4 Page faults**

- 0 is already there so **0 Page fault**.
- When 3 came it will take the place of 7 because it is least recently used **1 Page fault**
- 0 is already in memory so **0 Page fault**.
- 4 will takes place of 1, **1 Page Fault**
- Now for the further page reference string **0 Page fault** because they are already available in the memory.

ALGORITHM:

1. Start
2. Read the number of frames
3. Read the number of pages
4. Read the page numbers
5. Initialize the values in frames to -1
6. Allocate the pages in to frames by selecting the page that has not been used for the longest period of time.
7. Display the number of page faults.
8. Stop

3. OPTIMAL Algorithm.**THEORY:**

In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

Example:

Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, and 3 with 4 page frame. Find number of page fault.

Page reference	7,0,1,2,0,3,0,4,2,3,0,3,2,3														No. of Page frame - 4	
7	0	1	2	0	3	0	4	2	3	0	3	2	3			
			2	2	2	2	2	2	2	2	2	2	2			
		1	1	1	1	1	4	4	4	4	4	4	4			
	0	0	0	0	0	0	0	0	0	0	0	0	0			
7	7	7	7	7	3	3	3	3	3	3	3	3	3			
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit			
Total Page Fault = 6																

- Initially all slots are empty, so when 7 0 1 2 are allocated to the empty slots, **4 Page faults**
- 0 is already there so **0 Page fault**.
- When 3 came it will take the place of 7 because it is not used for the longest duration of time in the future, **1 Page fault**.
- 0 is already there so **0 Page fault**.
- 4 will takes place of 1, **1 Page Fault**.
- Now for the further page reference string **0 Page fault** because they are already available in the memory.

Optimal page replacement is perfect, but not possible in practice as the operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analyzed against it.

ALGORITHM:

1. Start the program
2. Read the number of frames
3. Read the number of pages
4. Read the page numbers
5. Initialize the values in frames to -1
6. Allocate the pages in to frames by selecting the page that will not be used for the longest period of time.
7. Display the number of page faults.
8. Stop the program

EXPERIMENT 10**DISK SCHEDULING****AIM:**

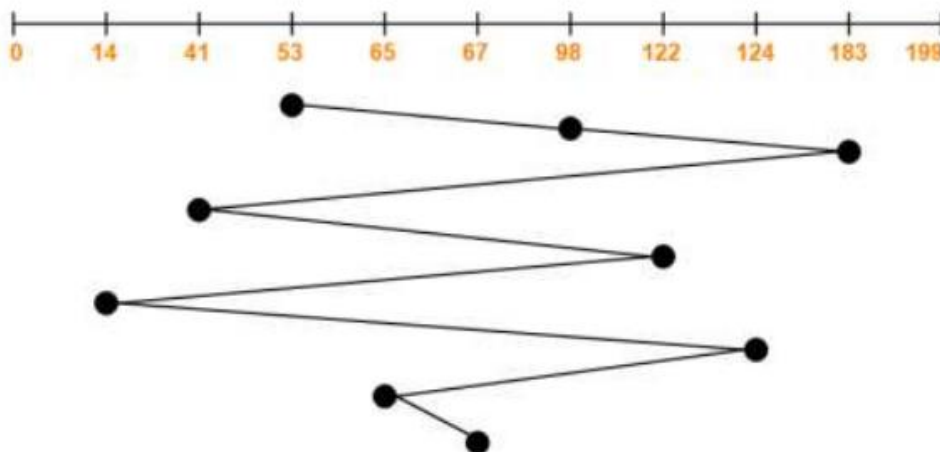
Implementation of Disk Scheduling using FCFS, SCAN, C-SCAN, LOOK, C-LOOK and SSTF algorithm

1. First Come First Serve (FCFS)**THEORY**

This algorithm entertains requests in the order they arrive in the disk queue.

Example:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53.

**ALGORITHM**

1. Start the program.
2. Mark the 'head' as the initial position of disk head.
3. Let request array represent an array storing indexes of track that have been requested in ascending order of their time of arrival.
4. One by one take the tracks in default order and calculate the absolute distance of the track from the head.
5. Increment the total seeks count with this distance.
6. New head position is currently serviced track position.
7. Go to step no. 3 until all track in request array have not been serviced.

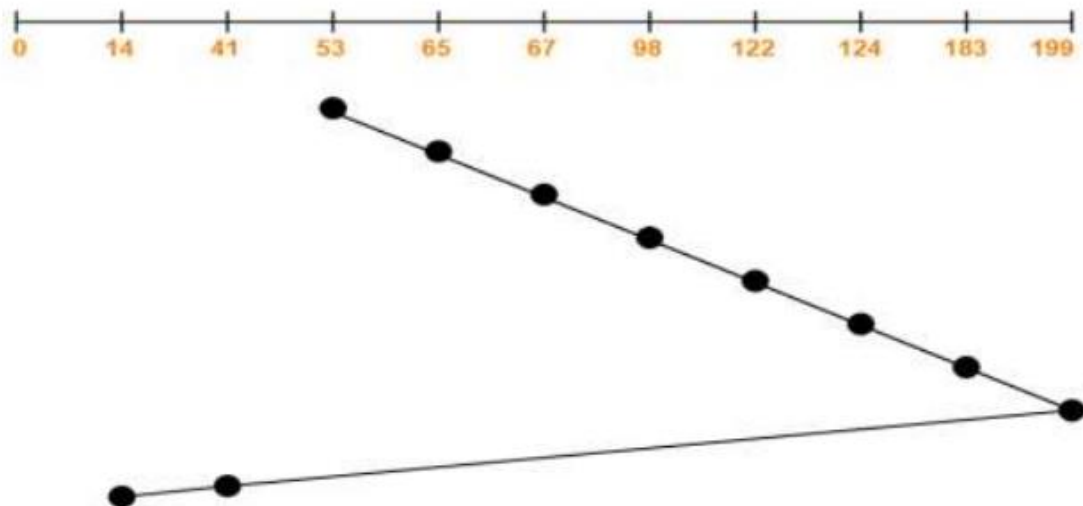
2. SCAN Algorithm

THEORY

This algorithm scans all the cylinders of the disk back and forth. Head start from one end of the disk and move towards the other end servicing all the request in between. After reaching the other end, head reverses its direction and move towards the starting end servicing all the requests in between. The same process repeats.

Example:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53.



ALGORITHM

1. Start the program.
2. Mark the 'head' as the initial position of disk head.
3. Let request array represent an array storing indexes of track that have been requested in ascending order of their time of arrival.
4. Let direction represents whether the head is moving towards left or right.
5. In the direction in which head is moving service all tracks one by one.
6. Calculate the absolute distance of the track from the head.
7. Increments the total seek count with this distance.
8. Currently serviced track position now becomes the new head position.
9. Go to step 5 until we reach at one of the ends of the disk.
10. If reach at the end of the disk reverse the direction and go to step 4 until all tracks in request array have not been serviced.

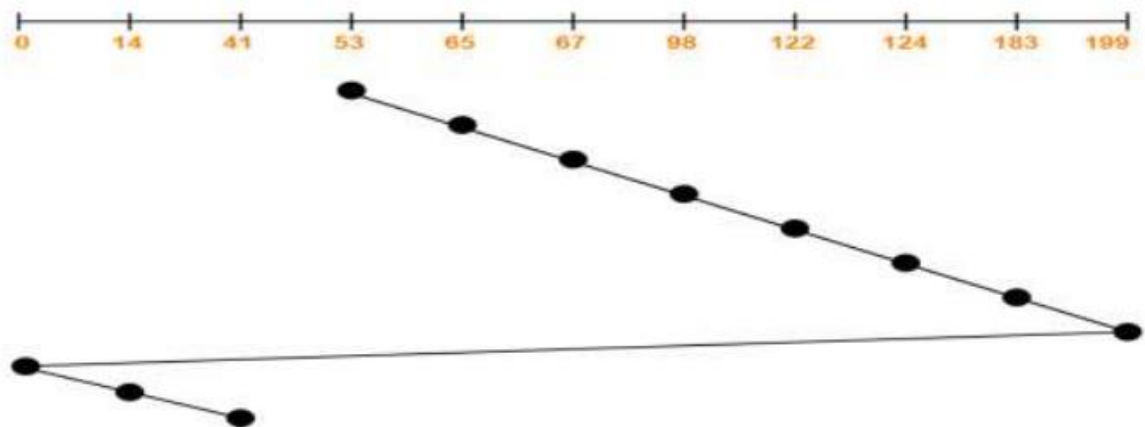
3. Circular- SCAN (C-SCAN) Algorithm

THEORY

Circular-SCAN Algorithm is an improved version of the SCAN. Head starts from one end of the disk and move towards the other end servicing all the requests in between. After reaching the other end, head reverses its direction. It then returns to the starting end without servicing any request in between. The same process repeats.

Example:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53.



ALGORITHM

1. Start the program.
2. Mark the 'head' as the initial position of disk head.
3. Let request array represent an array storing indexes of track that have been requested in
4. ascending order of their time of arrival.
5. The head services only in the right direction from 0 to the size of the disk.
6. While moving in the left directions do not service any of the tracks.
7. When we reach the beginning (left end) reverse the direction.
8. While moving in the right direction it services all tracks one by one.
9. While moving in the right directions calculate the absolute distance of the track from the head.
10. Increment the total seeks count with this distance.
11. Currently serviced track position now becomes the new head position.
12. Go to step 8 until we reach the right end of the disk.

13. If we reach the right end of the disk reverse the direction and go to step 5 until all tracks in the request array have not been serviced.

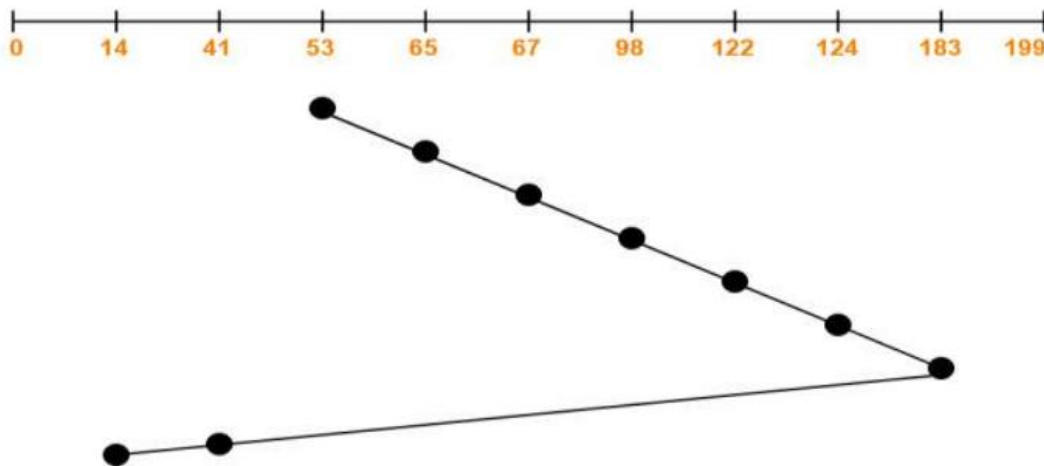
4. LOOK Algorithm

THEORY

LOOK Algorithm is an improved version of the SCAN Algorithm. Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between. After reaching the last request at the other end, head reverses its direction. It then returns to the first request at the starting end servicing all the requests in between. The same process repeats.

Example:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53.



ALGORITHM

1. Start the program.
2. Mark the 'head' as the initial position of disk head.
3. Let request array represent an array storing indexes of track that have been requested in ascending order of their time of arrival.
4. The initial direction in which head is moving is given and it services in the same direction.
5. The head services all the requests one by one in the direction head is moving.
6. The head continues to move in the same direction until the entire request in this direction are finished.
7. While moving in this direction calculates the absolute distance of the track from the head.
8. Increment the total seeks count with this distance.
9. Currently serviced track position now becomes the new head position.
10. Go to step 7 until we reach at last request in this direction.

11. If we reach where no requests are needed to be serviced in this direction reverse the direction and go to step 5 until all tracks in request array have not been serviced.

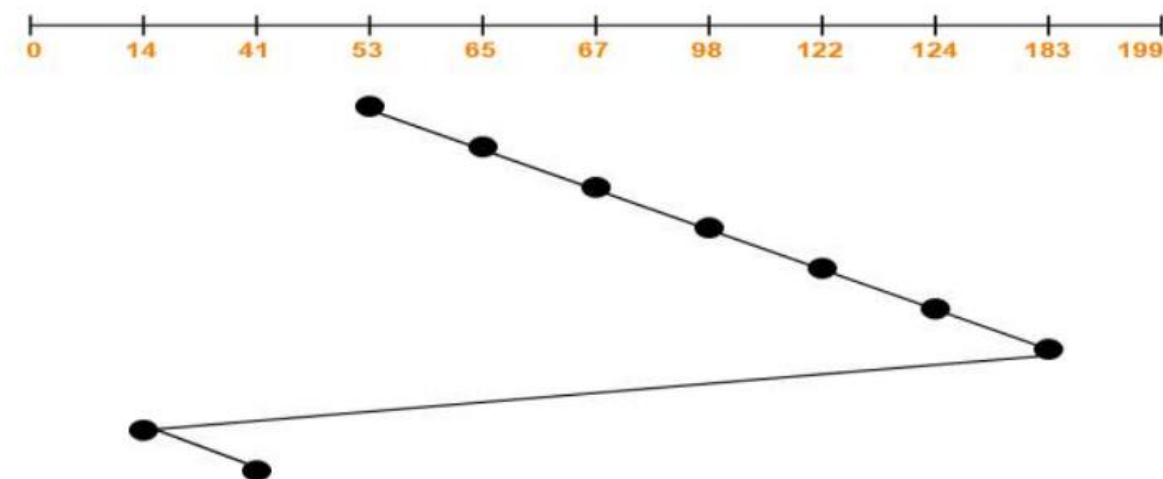
5. C-LOOK Algorithm

THEORY

Circular-LOOK Algorithm is an improved version of the LOOK Algorithm. Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between. After reaching the last request at the other end, head reverses its direction. It then returns to the first request at the starting end without servicing any request in between. The same process repeats.

Example:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53.



ALGORITHM

1. Start the program.
2. Mark the 'head' as the initial position of disk head.
3. Let request array represent an array storing indexes of track that have been requested in ascending order of their time of arrival.
4. The initial direction in which the head is moving is given and it services in the same direction.
5. The head services all the requests one by one in the direction it is moving.
6. The head continues to move in the same direction until all the requests in this direction have been serviced.
7. While moving in this direction, calculate the absolute distance of the tracks from the head.
8. Increment the total seeks count with this distance.

9. Currently serviced track position now becomes the new head position.
10. Go to step 5 until we reach the last request in this direction.
11. If we reach the last request in the current direction then reverse the direction and move the head in this direction until we reach the last request that is needed to be serviced in this direction without servicing the intermediate requests.
12. Reverse the direction and go to step 3 until all the requests have not been serviced.

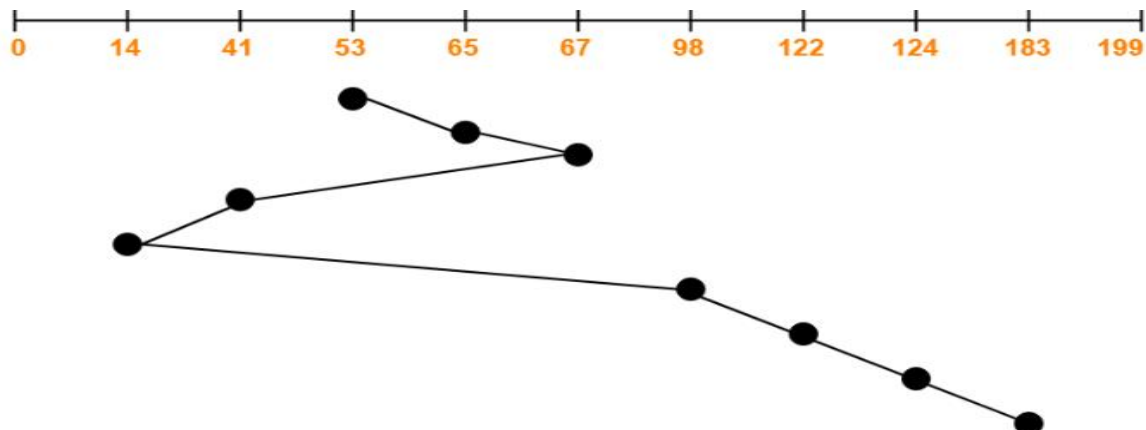
6. Shortest Seek Time First (SSTF)

THEORY

SSTF stands for Shortest Seek Time First. This algorithm services that request next which requires least number of head movements from its current position regardless of the direction. It breaks the tie in the direction of head movement.

Example:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53



ALGORITHM

1. Start the program.
2. Mark the 'head' as the initial position of disk head.
3. Let request array represent an array storing indexes of track that have been requested in ascending order of their time of arrival.
4. Find the positive distance of all tracks in the request array from head.
5. Find a track from requested array which has not been accessed/serviced yet and has minimum distance from head.
6. Increments the total seek count with this distance.
7. Currently serviced track position now becomes the new head position.
8. Go to step 4 until all tracks in request array have not been serviced.

EXPERIMENT -11

FILE ORGANIZATION TECHNIQUES & ALLOCATION STRATEGY

AIM:

Implement File Organization Technique (Single Level and Multi-Level Directory) and Allocation Strategies (Sequential, Linked and Indexed)

FILE ORGANIZATION TECHNIQUES

A) SINGLE LEVEL DIRECTORY:

THEORY

The directory structure is the organization of files into a hierarchy of folders. In a single-level directory system, all the files are placed in one directory. There is a root directory which has all files. It has a simple architecture and there are no sub directories. Advantage of single level directory system is that it is easy to find a file in the directory.

ALGORITHM:

- Step-1: Start the program.
- Step-2: Declare the count, file name, graphical interface.
- Step-3: Read the number of files
- Step-4: Read the file name
- Step-5: Declare the root directory
- Step-6: Using the file eclipse function define the files in a single level
- Step-7: Display the files
- Step-8: Stop the program

B) TWO LEVEL DIRECTORY

THEORY

In the two-level directory system, each user has own user file directory (UFD). The system maintains a master block that has one entry for each user. This master block contains the addresses of the directory of the users. When a user job starts or a user logs in, the system's master file directory (MFD) is searched. When a user refers to a particular file, only his own UFD is searched.

ALGORITHM:

- Step-1: Start the program.
- Step-2: Declare the count, file name, graphical interface.
- Step-3: Read the number of files
- Step-4: Read the file name
- Step-5: Declare the root directory

Step-6: Using the file eclipse function define the files in a single level

Step-7: Display the files

Step-8: Stop the program

FILE ALLOCATION STRATEGIES

A) SEQUENTIAL:

THEORY

The most common form of file structure is the sequential file in this type of file, a fixed format is used for records. All records (of the system) have the same length, consisting of the same number of fixed length fields in a particular order because the length and position of each field are known, only the values of fields need to be stored, the field name and length for each field are attributes of the file structure.

ALGORITHM

Step 1: Start the program.

Step 2: Get the number of files.

Step 3: Get the memory requirement of each file.

Step 4: Allocate the required locations to each in sequential order

Randomly select a location from available location $s1 = \text{random}(100)$;

Check whether the required locations are free from the selected location.

B) INDEXED:

THEORY

In the chained method file allocation table contains a field which points to starting block of memory. From it for each block a pointer is kept to next successive block. Hence, there is no external fragmentation.

ALGORITHM:

Step 1: Start the program.

Step 2: Get the number of files.

Step 3: Get the memory requirement of each file.

Step 4: Allocate the required locations by selecting a location randomly $q = \text{random}(100)$;

a) Check whether the selected location is free.

b) If the location is free allocate and set $\text{flag} = 1$ to the allocated locations.

C) LINKED:

THEORY

In the chained method file allocation table contains a field which points to starting block of memory. From it for each block a pointer is kept to next successive block. Hence, there is no external fragmentation

ALGORITHM:

Step 1: Start the program.

Step 2: Get the number of files.

Step 3: Get the memory requirement of each file.

Step 4: Allocate the required locations by selecting a location randomly $q = \text{random}(100)$;

a) Check whether the selected location is free.

b) If the location is free allocate and set $\text{flag} = 1$ to the allocated locations. While allocating next location address to attach it to previous location

EXPERIMENT- 12

Cloud Platforms and Virtualization tools

AIM:

Familiarization of Cloud Platforms and Virtualization tools (eg:-VMWare)

THEORY

Cloud Computing

"The cloud" refers to servers that are accessed over the Internet and the software and databases that run on those servers. Cloud servers are in data centres all over the world. By using cloud computing, users and companies do not have to manage physical servers themselves or run software applications on their own machines.

The cloud enables users to access the same files and applications from almost any device, because the computing and storage takes place on servers in a data centre, instead of locally on the user device. Therefore, a user can log into their Instagram account on a new phone after their old phone breaks and still find their old account in place, with all their photos, videos, and conversation history. It works the same way with cloud email providers like Gmail or Microsoft Office 365, and with cloud storage providers like Drop box or Google Drive.

The term “**Cloud Computing**” refers to services provided by the cloud that is responsible for delivering of computing services such as servers, storage, databases, networking, software, analytics, intelligence, and more, over the Cloud (Internet). Cloud computing applies a virtualized platform with elastic resources on demand by provisioning hardware, software, and data sets dynamically

Types

Cloud computing services are divided into three classes, according to the abstraction level of the capability provided and the service model of providers, namely:

1. Infrastructure as a Service,
2. Platform as a Service,
3. Software as a Service.

Infrastructure as a Service

A cloud infrastructure enables on-demand provisioning of servers running several choices of operating systems and a customized software stack. Infrastructure services are considered as the bottom layer of cloud computing systems. Offering virtualized resources (computation, storage, and communication) on demand is known as Infrastructure as a Service (IaaS).

Platform as a Service

A cloud platform offers an environment on which developers create and deploy applications and do not necessarily need to know how many processors or how much memory that applications will be using. In addition, multiple programming models and specialized services (e.g., data access, authentication, and payments) are offered as building blocks to new applications.

Software as a Service

Traditional desktop applications such as word processing and spreadsheet can now be accessed as a service in the Web. This model of delivering applications, known as Software as a Service (SaaS), alleviates the burden of software maintenance for customers and simplifies development and testing for providers.

Virtualization tools -VMWare

Virtualization

Virtualization is a way to use one computer as if it were many. Before virtualization, most computers were only doing one job at a time, and a lot of their power was wasted. Virtualization lets you run several virtual computers on one real computer, so you can use its full power and do more tasks at once.

In cloud computing, this idea is taken further. Cloud providers use virtualization to split one big server into many smaller virtual ones, so businesses can use just what they need, no extra hardware, no extra cost.

Virtualizations uses special software known as hypervisor, to create many virtual computers (cloud instances) on one physical computer. The Virtual Machines behave like actual computers but use the same physical machine.

A hypervisor is the software that gets virtualization to work. It serves as an intermediary between the physical computer and the virtual machines. The hypervisor controls the virtual machines' use of the physical resources (such as the CPU and memory) of the host computer.

VMware

First major virtualization software, developed by VMware Inc., USA in the mid-1990s. Leader in server and desktop virtualization.

Free Products by VMware

- **VMware Player:** End-user software that runs existing VMs but cannot create new ones; limited OS support and features.
- **VMware Server:** Designed for small businesses; can create and run VMs, suitable for low-end servers; similar to Workstation but less detailed here.

- **VMware Workstation:** Flagship product allowing creation, running, and importing of VMs, including physical machine virtualization; offers premium features like unlimited snapshots and tutorial creation; costs about \$60 new, \$50 upgrade.
- **VMware Academic Alliance:** Free online program offering unlimited classroom licenses to US educational institutions.

VMware Server Architecture:

- Installs on top of an existing OS, not bare-metal.
- Virtualization software sits between host OS and guest OS.

Cons:

- Additional OS layer reduces performance by consuming hardware resources.
- Virtualization competes with other host applications for hardware scheduling.
- Offers fewer features than VMware ESX Server (e.g., no failover, limited VM management).

Pros:

- No special hardware configuration needed.
- Compatible with almost any existing hardware.
- Easier to deploy on individual machines compared to ESX Server.